

Algebra of Programming

Bird & de Moor, chapters 5–10 — in Freyd diagram order

Composition is juxtaposition in diagram order: RS is “first R , then S ”, so every composite here is the reverse of the book’s. A row tagged ✓ has its mirrored statement machine-checked under AOP/A*.lean.

5 Datatypes in Allegories

5.1 Relators

- 5.1 ✓ **RELATOR** $F : \mathcal{A} \rightarrow \mathcal{B}$ between tabular allegories $\triangleq$ a *monotonic* functor: $R \subset S \implies FR \subset FS$.
- Lem 5.1 ✓ F a relator and f a map $\implies Ff$ is a map and $(Ff)^\circ = F(f^\circ)$.
- Thm 5.1 ✓ A functor is a relator $\iff$ it preserves converse: $F(R^\circ) = (FR)^\circ$.
- Cor 5.1 ✓ Relators agreeing on maps are equal: $(\forall f. Ff = Gf) \implies F = G$. Tabulate: $FR = F(f^\circ g) = (Ff)^\circ (Fg)$.
- Hence FR° is unambiguous — it names $(FR)^\circ$ and $F(R^\circ)$ alike.
- Ex 5.2 ✓ A relator preserves meets of coreflexives, $F(C \cap D) = FC \cap FD$. The restriction $C, D \subset 1$ is needed.
- Ex 5.5 ✓ $F(\text{dom } R) = \text{dom}(FR)$.
- Ex 5.4 Not every functor on $\mathbf{Fun}$ extends to a relator: for $FA = \emptyset$ if $A = \emptyset$ and $\{0\}$ otherwise, the constant maps one, two : $\{0\} \rightarrow \{1, 2\}$ give $F(\text{two one}^\circ) = 1$, while preservation of converse forces $\emptyset$.

5.2 Relational products

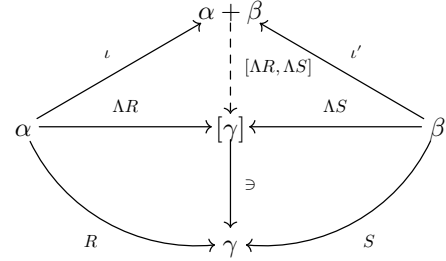
(π, π') tabulates Π , so $\langle f, g \rangle = (f\pi^\circ) \cap (g\pi'^\circ)$ for maps; (5.1) reads that same formula for arbitrary relations.

- (5.1) ✓ $\langle R, S \rangle \triangleq (R\pi^\circ) \cap (S\pi'^\circ)$. Rel $c \langle R, S \rangle (a, b) \iff cRa \wedge cSb$.
- (5.2) ✓ $R \times S \triangleq \langle \pi R, \pi' S \rangle$.
- $\times$ is monotonic in both arguments and $(R \times S)^\circ = R^\circ \times S^\circ$, so it is a relator — but it is *not* a categorical product.
- (5.3) ✓ **ABSORPTION** $\langle X, Y \rangle (R \times S) = \langle XR, YS \rangle$; hence $(R \times S)(U \times V) = (RU) \times (SV)$.
- (5.4) ✓ $\langle XR, Y \rangle \subset \langle X, Y \rangle (R \times 1)$ — one of the two special cases (5.3) is staged through.
- (5.5) ✓ $\langle X, YS \rangle \subset \langle X, Y \rangle (1 \times S)$, symmetric to (5.4).
- (5.6) ✓ $\langle R, S \rangle \pi = (\text{dom } S)R$.
- (5.7) ✓ $\langle R, S \rangle \pi' = (\text{dom } R)S$.
- So $\langle R, \emptyset \rangle \pi = \emptyset$, not R : precisely why (π, π') is no categorical product in an allegory.
- (5.8) ✓ **CANCELLATION** $\langle X, Y \rangle \langle R, S \rangle^\circ = (XR^\circ) \cap (YS^\circ)$.
- Ex 5.6 ✓ $(1 \times S)(R \times 1) \supset R \times S$, from (5.4) and (5.2) alone.
- Ex 5.7 $\langle R, S \rangle^\circ (P, Q) \subset (R^\circ P) \times (S^\circ Q)$.
- Ex 5.8 $\langle X, Y \rangle (R \times S) \subset \langle XR, YS \rangle$ — the easy half of (5.3).
- Ex 5.9 ✓ $\langle fR, fS \rangle = \langle fR, fS \rangle$ for a map f ; false for an arbitrary arrow.
- Ex 5.10 $\text{unzip}(F) \triangleq \langle F\pi, F\pi' \rangle$, and $F(R \times S) \text{unzip}(F) = \text{unzip}(F)(FR \times FS)$.

5.3 Relational coproducts

A coproduct of α, β in $\mathbf{Fun}(\mathcal{A})$ is a coproduct in the whole power allegory $\mathcal{A}$:

$$\iota T = R \wedge \iota' T = S \iff T = [\Lambda R, \Lambda S] \exists.$$



- 5.3 ✓ **JUNC** $[R, S] \triangleq [\Lambda R, \Lambda S] \exists$.
- (5.9) ✓ $[R, S] = (\iota^\circ R) \cup (\iota'^\circ S)$.
- (5.10) ✓ $R + S \triangleq [R\iota, S\iota']$, monotonic in both arguments, so $+$ is a relator.
- (5.11) ✓ **CANCELLATION** $[U, V]^\circ [R, S] = (U^\circ R) \cup (V^\circ S)$. Easier than (5.8) — but still not the dual of it, an allegory being its own opposite.
- Ex 5.12 ✓ $\iota[1, \emptyset] = 1$ and $\iota'[1, \emptyset] = \emptyset$, and $[1, \emptyset]\iota \cup [\emptyset, 1]\iota' = [\iota, \iota'] = 1$; hence $[1, \emptyset] = \iota^\circ$, $[\emptyset, 1] = \iota'^\circ$, which gives (5.9).
- Ex 5.14 $(R + S) \cap ([P, Q][U, V]^\circ) = (R \cap PU^\circ) + (S \cap QV^\circ)$.

5.4 The power relator

- 5.4 ✓ On maps $Pf = ((\exists f)/\exists) \cap (\exists^\circ \setminus (f\exists^\circ))$; read as a definition for relations,
- $$PR \triangleq ((\exists R)/\exists) \cap (\exists^\circ \setminus (R\exists^\circ)).$$
- Rel $x (PR) y \iff (\forall a \in x. \exists b \in y. aRb) \wedge (\forall b \in y. \exists a \in x. aRb)$. Every element of x has an R -partner in y , and conversely.
- ✓ P is monotone in R , straight from monotonicity of division.
- ✓ $P1 = 1$ — this is the antisymmetry of subset $= \exists/\exists$ restated, so it needs the allegory unitary and tabular.
- ✓ $(PR)(PS) = P(RS)$, so P is a relator. $\subset$ is division cancellation; $\supset$ needs a tabulation (x, z) of $P(RS)$ and the mediating map $y = \Lambda((x\exists R^\circ) \cap (z\exists S))$.
- ✓ P and E agree on maps: $Pf = \Lambda(\exists f) = Ef$.
- Ex 5.15 They differ on relations, and Corollary 5.1 does not object: E is not monotonic — distinct maps are never comparable, so $R \subset S$ says nothing about $\Lambda(\exists R)$ and $\Lambda(\exists S)$ — hence not a relator.
- Ex 5.16 $P(\text{dom } R)(ER) \subset PR$.

5.5 Relational catamorphisms

Let F have initial algebra in : $FT \rightarrow T$ in the subcategory of maps. It is then initial for relational algebras as well.

- (5.12) ✓ in $X = (FX)R \iff X = (\Lambda((F\exists)R))\exists$.
- 5.5 ✓ $\llbracket R \rrbracket \triangleq (\Lambda((F\exists)R))\exists$ — the unique X with in $X = (FX)R$.
- 5.5 ✓ **EILENBERG–WRIGHT** $\Lambda\llbracket R \rrbracket = \llbracket \Lambda((F\exists)R) \rrbracket$: the transpose of a relational catamorphism is a catamorphism of maps. This is what turns a

relational specification into a functional fold over sets, and it is used throughout §5.6.

- ✓ Over a map algebra the relational catamorphism is the ordinary one.

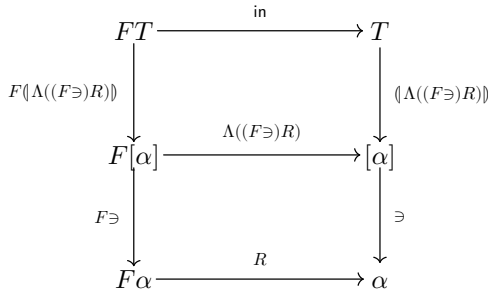
5.5 **TYPE RELATOR** F binary with initial type (α, T) : then $(TR)^\circ = T(R^\circ)$, so T is a relator. By uniqueness — both sides solve in $X = F(R^\circ, X)$ in.

Ex 5.17 ✓ In a Boolean allegory each coreflexive C has a $\sim C$ with $C \cap \sim C = \emptyset$ and $C \cup \sim C = 1$. Then guard $C \triangleq [C, \sim C]^\circ$ is a map, $(C \rightarrow R, S) \triangleq (\text{guard } C)(R + S)$, and conditionals behave like cases:

$$R \subset (C \rightarrow S, T) \iff CR \subset S \wedge (\sim C)R \subset T.$$

Ex 5.18 F preserves meets $\implies$ so does the type relator T it induces; in particular list does.

Ex 5.19 R entire $\implies \langle R \rangle$ entire (reflection gives $\text{dom } \langle R \rangle = 1$).



5.6 Combinatorial functions

Lists mean cons-lists, and list is their type functor, list $R = \langle \text{nil}, (R \times 1) \text{ cons} \rangle$. Each relation below is specified as a catamorphism, transposed by Eilenberg–Wright, then implemented on lists.

Subsequences

5.6 $\text{subseq} = \langle \langle \text{nil}, \text{cons} \cup \pi' \rangle \rangle : \text{list } \alpha \rightarrow \text{list } \alpha$ — at each step keep the head or drop it.

$\Lambda \text{ subseq} = \langle \text{nil } \tau, \langle \Lambda(1 \times \triangleright)(P \text{ cons}), \pi' \rangle \text{ cup} \rangle$, by expanding $\Lambda(\langle \text{nil}, \text{cons} \cup \pi' \rangle F(1, \triangleright))$ for $F(A, B) = 1 + (A \times B)$. Pointwise: $e = \{\emptyset\}$ and $f(a, xs) = \{\text{cons}(a, x) \mid x \in xs\} \cup xs$.

$\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle \text{ cup}$, where cup unions a pair of sets.

5.6 Sets replaced by lists: $\text{subseqs} = \langle \text{nil wrap}, \langle \text{cpr}(\text{list cons}), \pi' \rangle \text{ cat} \rangle$, i.e. $e = [\emptyset]$ and $f(a, xs) = [\text{cons}(a, x) \mid x \leftarrow xs] \# xs$.

$\text{cpr} : \alpha \times \text{list } \beta \rightarrow \text{list}(\alpha \times \beta)$ (“cartesian product, right”), $\text{cpr}(a, x) = [(a, b) \mid b \leftarrow x]$.

5.6 $\text{setify} \triangleq \langle \omega, (\tau \times 1) \text{ cup} \rangle : \text{list } \alpha \rightarrow [\alpha]$, a list’s set of elements; ω returns the empty set.

Ex 5.21 $\text{nil setify} = \omega$, $\text{wrap setify} = \tau$, $\text{cat setify} = (\text{setify} \times \text{setify}) \text{ cup}$, $(\text{list } f) \text{ setify} = \text{setify } (Pf)$.

$\text{concat setify} = (\text{list setify}) \text{ setify} \cup$, $\text{cpr setify} = (1 \times \text{setify}) \Lambda(1 \times \triangleright)$.

Ex 5.22 $\text{subseqs setify} = \Lambda \text{ subseq}$ — the list program implements the set specification. By fusion, from the identities above.

Ex 5.23 subseq is the converse of a catamorphism, the one computing supersequences.

Cartesian product

5.6 $\text{cpp}(x, y) = [(a, b) \mid a \leftarrow x, b \leftarrow y]$ and $\text{cpl}(x, b) = [(a, b) \mid a \leftarrow x]$, with $\text{cpp setify} = \Lambda(\triangleright \times \triangleright)$ and $\text{cpl setify} = \Lambda(\triangleright \times 1)$.

5.6 $\text{cp}(F) \triangleq \Lambda(F \triangleright) : F[\alpha] \rightarrow [F\alpha]$ — one choice from each slot. cpr , cpp , cpl implement it for $FA = \alpha \times A$, $A \times A$ and $A \times \beta$.

$\text{cp}(\text{list}) = \langle \Lambda[\text{nil}, (\triangleright \times \triangleright) \text{ cons}] \rangle = \langle \text{nil } \tau, \Lambda(\triangleright \times \triangleright)(P \text{ cons}) \rangle$ by Eilenberg–Wright;
 $\text{cp}(\text{list})[x_1, \dots, x_n] = \{[a_1, \dots, a_n] \mid a_j \in x_j\}$. On lists: $\text{cplst} = \langle \text{nil wrap}, \text{cpp}(\text{list cons}) \rangle$.

Ex 5.20 ✓ $\Lambda(R \cup S) = \langle \Lambda R, \Lambda S \rangle \text{ cup}$, $\Lambda(R \cap S) = \langle \Lambda R, \Lambda S \rangle \text{ cap}$, $\Lambda(R \times S) = (\Lambda R \times \Lambda S) \text{ cross}$, with $\text{cup} = \Lambda((\pi \triangleright) \cup (\pi' \triangleright))$ and $\text{cap} = \Lambda((\pi \triangleright) \cap (\pi' \triangleright))$.

Prefix and suffix

5.6 $\text{prefix} = \text{cat}^\circ \pi$: $x \text{ prefix } y$ when $x \# z = y$ for some z . Also $\text{prefix} = \text{init}^*$ with $\text{init} = \text{snoc}^\circ \pi$, and $\text{prefix} = \langle \text{nil}, \text{nil} \cup \text{cons} \rangle$ — the second nil being $! \text{nil} : \alpha \times \text{list } \alpha \rightarrow \text{list } \alpha$.

$\Lambda \text{ prefix} = \langle \text{nil } \tau, \langle \text{nil } \tau, \Lambda(1 \times \triangleright)(P \text{ cons}) \rangle \text{ cup} \rangle$.

5.6 $\text{inits} = \langle \text{nil wrap}, \langle \text{nil wrap}, \text{cpr}(\text{list cons}) \rangle \text{ cat} \rangle$, i.e. $\text{inits } [] = [[]]$ and $\text{inits } ([a] \# x) = [[]] \# [[a] \# y \mid y \leftarrow \text{inits } x]$ — initial segments in increasing order of length.

5.6 $\text{suffix} = \text{tail}^*$ with $\text{tail} = \text{cons}^\circ \pi'$, the snoc-list dual of prefix. $\text{tails } [] = [[]]$ and $\text{tails } (x \# [a]) = [y \# [a] \mid y \leftarrow \text{tails } x] \# [[]]$ — decreasing length.

Not implementable as it stands; as a cons-list catamorphism $\text{tails} = \langle \text{nil wrap}, \text{extend} \rangle$ with $\text{extend}(a, [x] \# xs) = [[a] \# x] \# [x] \# xs$.

5.6 $\text{splits} = \langle \text{inits}, \text{tails} \rangle \text{ zip}$ implements $\Lambda(\text{cat}^\circ)$, every way of splitting a list in two — and works only because inits and tails order their segments oppositely.

Ex 5.24 $\text{init} : \text{list}^+ \alpha \rightarrow \text{list } \alpha$ is itself a catamorphism.

Partitions

5.6 $\text{partition} \triangleq \text{concat}^\circ : \text{list } \alpha \rightarrow \text{list}(\text{list}^+ \alpha)$ — cut a list into non-empty contiguous segments. Rel $\text{partition } [1, 2, 3] = \{[[1], [2], [3]], [[1, 2], [3]], [[1], [2, 3]], [[1, 2, 3]]\}$.

As a catamorphism $\text{partition} = \langle \text{nil}, \text{new} \cup \text{glue} \rangle$ with $\text{new} = (\text{wrap} \times 1) \text{ cons}$ and $\text{glue} = (1 \times \text{cons}^\circ) \text{ assoc}(\text{cons} \times 1) \text{ cons}$: at each step either open a segment or glue the element onto the first one. Pointwise $\text{new}(a, xs) = [[a]] \# xs$ and $\text{glue}(a, [x] \# xs) = [[a] \# x] \# xs$.

6 The two agree by the converse-catamorphism theorem of the next chapter: $\text{ran } R = 1$ and $Rf \subset F(1, f) \text{ in} \implies f^\circ = \langle R \rangle$, applied to $f = \text{concat}$ and $R = [\text{nil}, \text{new} \cup \text{glue}]$.

Ex 5.25 Its surjectivity rests on $\text{new}^\circ \text{new} = \text{cons}^\circ(\text{wrap}^\circ \text{ wrap} \times 1) \text{ cons}$ and $\text{glue}^\circ \text{glue} = \text{cons}^\circ(\text{cons}^\circ \text{ cons} \times 1) \text{ cons}$.

$\Lambda \text{ partition} = \langle \Lambda \text{ nil}, \Lambda((1 \times \triangleright)(\text{new} \cup \text{glue})) \rangle$, whose step simplifies (Ex 5.27) to $\Lambda(1 \times \triangleright) P(\langle \text{new } \tau, \Lambda \text{ glue} \rangle \text{ cup}) \cup$.

5.6 $\text{partitions} = \langle \text{nil wrap}, \text{cpr}(\text{list } (\langle \text{new}, \text{glue} \rangle \text{ cons})) \text{ concat} \rangle$, where glue implements $\Lambda \text{ glue}$: $\text{glue}(a, []) = []$ and $\text{glue}(a, [x] \# xs) = [[[a] \# x] \# xs]$.

Ex 5.26 $\text{partitions setify} = \Lambda \text{ partition}$.

Permutations

5.6 $\text{bag } \alpha$ is the initial algebra $[\text{bnil}, \text{bcons}]$ of $F(A, B) = 1 + A \times B$ whose bcons commutes, $(1 \times \text{bcons}) \text{ bcons} = \text{exch } (1 \times \text{bcons}) \text{ bcons}$, i.e. $\text{bcons}(a, \text{bcons}(b, x)) = \text{bcons}(b, \text{bcons}(a, x))$: order is

irrelevant, duplicates are not. $\text{exch} = \text{assocl} (\text{swap} \times 1) \text{ assocr}$.

5.6 $\text{bagify} = \langle \text{bnil}, \text{bcons} \rangle : \text{list } \alpha \rightarrow \text{bag } \alpha$ is surjective, $\text{bagify}^\circ \text{ bagify} = 1$.

5.6 $\text{perm} \triangleq \text{bagify} \text{ bagify}^\circ$ — equal bags of values. At once $\text{perm} = \text{perm}^\circ$.

As a catamorphism $\text{perm} = \langle \text{nil}, \text{add} \rangle$ with $\text{add} = \text{cons perm}$, or with the usable $\text{add} = (1 \times \text{cat}^\circ) \text{ exch} (1 \times \text{cons}) \text{ cat}$: $\text{add}(a, x) = y \# [a] \# z$ where $y \# z = x$, so a goes in anywhere.

Ex 5.28 $\text{cons perm} = (1 \times \text{perm}) \text{ cons perm}$, from $\text{bagify}^\circ \text{ bagify} = 1$.

5.6 $\text{adds}(a, x) = [y \# [a] \# z \mid (y, z) \leftarrow \text{splits } x]$ and $\text{perms} = \langle \text{nil wrap}, \text{cpr} (\text{list add}) \rangle$.

Ex 5.29 interleave , all interleavings of two lists, is the converse of a catamorphism.

5.7 Lax natural transformations

Some equalities of the functional world become inequalities in the relational one.

(5.13) $\checkmark$ **LAX NATURAL** $\varphi : F \hookrightarrow G \triangleq \varphi(FR) \supset (GR)\varphi$ for every R , where $\varphi_\alpha : G\alpha \rightarrow F\alpha$. The $\supset$ is the way the hook points.

$$\begin{array}{ccc} G\alpha & \xrightarrow{\varphi} & F\alpha \\ GR \downarrow & \supset & \downarrow FR \\ G\beta & \xrightarrow{\varphi} & F\beta \end{array}$$

5.7 Example: $\exists : 1 \hookrightarrow P$, that is $(PR)\exists \subset \exists R$ — immediate from the definition of PR .

Thm 5.2 $\checkmark$ F, G relators: φ is lax natural $F \hookrightarrow G \iff \varphi$ is strictly natural on maps, $\varphi(Ff) = (Gf)\varphi$. So lax naturality adds nothing to naturality of the underlying map functors — it only records how it extends.

Ex 5.30 Three instances: $R\tau \subset \tau(PR)$, $R(1, 1) \subset \langle 1, 1 \rangle (R \times R)$, $(PR \times PR) \text{ cup} \subset \text{cup} (PR)$.

6 Recursive Programs

6.1 Digits of a number

Convert a positive natural to its decimal digits. The pattern of the whole chapter in one example: specify a function as a refinement of some relation, discover that the relation solves a recursion equation, then read the recursion off as a program.

6.1 $\text{Decimal} ::= \text{wrap Digit}^+ \mid \text{snoc}(\text{Decimal}, \text{Digit})$, so $([\text{wrap}, \text{snoc}], \text{Decimal})$ is the initial algebra of $FA = \text{Digit}^+ + (A \times \text{Digit})$.

$\text{val} = \langle \text{embed}, \text{op} \rangle : \text{Decimal} \rightarrow \text{Nat}^+$, with embed the inclusion of digits and $\text{op}(n, d) = 10n + d$.

(6.1) $\text{digits} \subset \text{val}^\circ$ — a *functional refinement* of val° , which is not yet known to be a map. This is the specification.

$$\begin{aligned} \text{val}^\circ &= \text{definition} \\ &= \langle \text{embed}, \text{op} \rangle^\circ \\ &= \text{catamorphisms} \\ &= ([\text{wrap}, \text{snoc}]^\circ (F \text{ val}) [\text{embed}, \text{op}])^\circ \\ &= \text{converse} \\ &= [\text{embed}, \text{op}]^\circ (F \text{ val}^\circ) [\text{wrap}, \text{snoc}] \\ &= \text{definition of } F \\ &= [\text{embed}, \text{op}]^\circ (1 + (\text{val}^\circ \times 1)) [\text{wrap}, \text{snoc}] \\ &= \text{coproduct} \\ &= [\text{embed}, \text{op}]^\circ [\text{wrap}, (\text{val}^\circ \times 1) \text{ snoc}] \\ &= \text{coproduct} \\ &= (\text{embed}^\circ \text{ wrap}) \cup (\text{op}^\circ (\text{val}^\circ \times 1) \text{ snoc}) \end{aligned}$$

6.1 $\checkmark$ So $\text{val}^\circ = (\text{embed}^\circ \text{ wrap}) \cup (\text{op}^\circ (\text{val}^\circ \times 1) \text{ snoc})$.

$\text{op}(n, d) = m \iff n = m \text{ div } 10 \wedge d = m \text{ mod } 10$, so op° is defined exactly for $m \geq 10$ and embed° exactly for $m < 10$: the two branches have disjoint domains, and the join becomes a conditional.

$\text{val}^\circ m = \text{wrap } m$ if $m < 10$, else $\text{snoc}(\text{val}^\circ(m \text{ div } 10), m \text{ mod } 10)$. The recursion terminates because $m \geq 10 \implies m \text{ div } 10 < m$, so val° is a total function and $\text{digits} = \text{val}^\circ$.

$\text{digits } m = [m]$ if $m < 10$, else $\text{digits}(m \text{ div } 10) \# [m \text{ mod } 10]$ — quadratic, because snoc on lists is linear.

With an accumulator, $\text{digits } m = f(m, [])$ where $f(m, x) = [m] \# x$ if $m < 10$, else $f(m \text{ div } 10, [m \text{ mod } 10] \# x)$ — linear, and correct at 0.

6.2 Least fixed points

T 6.1 $\checkmark$ **KNASTER-TARSKI** φ a monotonic mapping on the arrows $\alpha \rightarrow \beta$ of a locally complete allegory. Then $\varphi X \subset X$ and $\varphi X = X$ have the same least solution, written $(\mu X : \varphi X)$; dually $X \subset \varphi X$ and $X = \varphi X$ have the same greatest solution.

Proof: $(\mu X : \varphi X) = \bigcap \{X : \varphi X \subset X\}$. Local completeness is what makes that meet exist.

in is an isomorphism, so $\langle R \rangle$ is the unique — hence both least and greatest — solution of $X = \text{in}^\circ (FX)R$, and $\varphi X = \text{in}^\circ (FX)R$ is monotonic. Knaster–Tarski then gives:

(6.2) $\checkmark$ $\langle R \rangle \subset X \iff \text{in}^\circ (FX)R \subset X$.

(6.3) $\checkmark$ $X \subset \langle R \rangle \iff X \subset \text{in}^\circ (FX)R$.

(6.4) $\checkmark$ **FUSION**, $\langle T \rangle \subset \langle R \rangle S \iff (FS)T \subset RS$.

(6.5) ✓ **FUSION**, $\triangleright (\llbracket R \rrbracket) S \subset (\llbracket T \rrbracket) \Leftarrow RS \subset (FS)T$. Equality of the two hypotheses gives back the catamorphism fusion law of ch. 3.

Ex 6.3 ✓ **KLEENE** φ *continuous* — it preserves joins of ascending chains — then $(\mu X : \varphi X) = \bigcup \{\varphi^n \emptyset : 0 \leq n\}$.

Ex 6.4 ✓ **FIXED POINT INDUCTION** $(\mu X : \varphi X) \subset A \Leftarrow \varphi A \subset A$; by Kleene, $(\forall X. X \subset A \Rightarrow \varphi X \subset A)$ suffices too.

Ex 6.5 The least solution of $\varphi X \subset X$ satisfies $\varphi X = X$ — Lambek's lemma, for the containment order read as a category and φ as a functor on it.

Ex 6.7 ✓ $(\llbracket R \rrbracket) \subset (\llbracket S \rrbracket) \Leftarrow (F \llbracket S \rrbracket) R \subset (F \llbracket S \rrbracket) S$. It does *not* need $R \subset S$.

Ex 6.8 ✓ R **DIFUNCTIONAL** $\triangleq R = RR^\circ R$; the difunctional closure of R is a least fixed point.

6.3 Hylomorphisms

6.3 A **HYLOMORPHISM** is a catamorphism after the converse of one, $(\llbracket S \rrbracket)^\circ (\llbracket R \rrbracket)$ for $R : F\alpha \rightarrow \alpha$ and $S : F\beta \rightarrow \beta$. The initial type μF is the *intermediate data structure*: it is built by $(\llbracket S \rrbracket)^\circ$ and consumed by $(\llbracket R \rrbracket)$, and never appears in the answer.

T 6.2 ✓ **HYLOMORPHISM THEOREM**

$$(\llbracket S \rrbracket)^\circ (\llbracket R \rrbracket) = (\mu X : S^\circ(FX)R).$$

S° divides, FX conquers, R combines.

Proof in two halves: $(\llbracket S \rrbracket)^\circ (\llbracket R \rrbracket)$ is a fixed point (functors and the two catamorphism laws), and it is below every prefixed point — that half goes through division, $(\llbracket S \rrbracket)^\circ (\llbracket R \rrbracket) \subset X \Leftarrow (\llbracket R \rrbracket) \subset X/(\llbracket S \rrbracket)^\circ$, which is the standard move for least fixed points.

C 6.1 ✓ $FX = GX + HX$, so an F -algebra is a pair: $(\llbracket S_1, S_2 \rrbracket)^\circ (\llbracket R_1, R_2 \rrbracket) = (\mu X : (S_1^\circ(GX)R_1) \cup (S_2^\circ(HX)R_2))$.

in is an isomorphism and $(\llbracket \text{in} \rrbracket) = 1$, so every catamorphism and every converse of one is already a hylomorphism.

Ex 6.10 ✓ Hylomorphisms preserve simplicity: R and S simple $\Rightarrow (\llbracket S \rrbracket)^\circ (\llbracket R \rrbracket)$ simple.

6.4 Fast exponentiation and modulus

6.4 $\exp a = (\llbracket \text{one}, \text{mult } a \rrbracket) : \text{Nat} \rightarrow \text{Nat}$ encodes $a^0 = 1$ and $a^{b+1} = a \times a^b$ — $O(b)$ steps. Divide and conquer brings it to $O(\log b)$.

$\text{Bin} = \text{listl Bit}$, $\text{Bit} = \{0, 1\}$; $\text{convert} = (\llbracket \text{zero}, \text{shift} \rrbracket) : \text{Bin} \rightarrow \text{Nat}$ with $\text{shift}(n, d) = 2n + d$ reads a bit sequence as a number. It is simple.

$\exp a$
 $\supseteq$ convert simple, so $\text{convert}^\circ \text{convert} \subset 1$
 $\text{convert}^\circ \text{convert} (\exp a)$
 $=$ fusion, with $\text{op } a(n, d) = (d = 0 \rightarrow n^2, a \times n^2)$
 $\text{convert}^\circ (\llbracket \text{one}, \text{op } a \rrbracket)$
 $=$ Corollary 6.1
 $(\mu X : (\text{zero}^\circ \text{one}) \cup (\text{shift}^\circ (X \times 1)(\text{op } a)))$

6.4 ✓ The fusion step needs zero ($\exp a = \text{one}$) and shift ($\exp a = (\exp a \times 1)(\text{op } a)$). zero and shift have disjoint ranges, so the join is again a conditional:
 $\exp ab = 1$ if $b = 0$, else $\text{op } a(\exp a(b \text{ div } 2), b \text{ mod } 2)$.

6.4 ✓ Same idea for $a \text{ mod } b$: $\text{mod } b = (\llbracket \text{zero}, \text{succ } b \rrbracket)$ with $\text{succ } ba = (a = b - 1 \rightarrow 0, a + 1)$ costs $O(a)$; fusing through convert gives

$$\text{mod } b \supset (\mu X : (\text{zero}^\circ \text{zero}) \cup (\text{shift}^\circ (X \times 1)(\text{op } b)))$$

with $\text{op } b(r, d) = (n \geq b \rightarrow n - b, n)$ and $n = 2r + d$.

Conditions: $\text{zero} (\text{mod } b) = \text{zero}$ and $\text{shift} (\text{mod } b) = (\text{mod } b \times 1)(\text{op } b)$. The program is $\text{mod } ba = 0$ if $a = 0$; $\text{op } b(\text{mod } b(a \text{ div } 2), 0)$ if a even;
 $\text{op } b(\text{mod } b(a \text{ div } 2), 1)$ if a odd — $O(\log a)$.

6.5 Unique fixed points

A hylomorphism is the *least* fixed point of its recursion, not necessarily the only one.

6.5 On Nat take $X = (\llbracket \text{zero}, \text{positive} \rrbracket)^\circ (\llbracket \text{zero}, 1 \rrbracket)$ with $\text{positive} = \text{succ}^\circ \text{succ}$ the coreflexive on positives. Both catamorphisms describe the coreflexive on 0, so X is that. But its recursion $X = (\text{zero}^\circ \text{zero}) \cup (\text{positive } X)$ also has $X = 1$ as a solution.

$[\text{zero}, \text{positive}]^\circ$ and $[\text{zero}, 1]$ are both maps, yet the least solution is not even entire. Simplicity does carry over (Ex 6.10); entireness is what needs a hypothesis, and the hypothesis is that a membership relation be *inductive*.

Inductive and well-founded relations

6.5 ✓ $R : \alpha \rightarrow \alpha$ is **INDUCTIVE** $\triangleq$ for all X ,
 $X/R \subset X \Rightarrow \Pi \subset X$.

Pointwise: if $(\forall c. cRa \Rightarrow cXb) \Rightarrow aXb$ holds for all a, b , then X holds everywhere — exactly a proof by induction over R .

$<$ on Nat gives ordinary mathematical induction; $\text{tail} = \text{cons}^\circ \pi'$ gives induction over lists.

Ex 6.13 ✓ S inductive and $RR \subset SR \Rightarrow R$ inductive. Hence $R \subset S$ with S inductive $\Rightarrow R$ inductive, and S inductive $\Leftarrow S^+$ inductive, where $S^+ = (\mu X : S \cup (SX))$.

Ex 6.15 ✓ The meet of two inductive relations is inductive; the join need not be.

6.5 ✓ R is **WELL-FOUNDED** $\triangleq$ for all X , $X \subset RX \Rightarrow X = \emptyset$ — no infinite chain $a_0 R a_1 R \dots$. Inductive $\Rightarrow$ well-founded, and the converse holds in a Boolean allegory.

Ex 6.16 ✓ R well-founded $\Rightarrow fRf^\circ$ well-founded, for any map f .

Membership

6.5 A relator F may carry a **MEMBERSHIP** arrow $\text{mem}(F)_\alpha : F\alpha \rightarrow \alpha$ with $a \text{ mem } x$ exactly when a is an element of x . For the polynomial relators, the power relator and the type relators it exists:

$$\text{mem}(\text{Id}) = 1, \quad \text{mem}(K_\beta) = \emptyset, \quad \text{mem}(F + G) = [\text{mem}(F), \text{mem}(G)].$$

$$\text{mem}(F \times G) = (\pi \text{ mem}(F)) \cup (\pi' \text{ mem}(G)), \quad \text{mem}(F \circ G) = \text{mem}(F) \text{ mem}(G), \quad \text{mem}([\cdot]) = \exists.$$

Type relators, via sets: $\text{mem}(T) = \text{setify}(T) \ni$ with $\text{setify}(T) = \Lambda(\text{mem}(T))$ — Ex 6.17 gives a set-free alternative.

mem is a lax natural transformation $1 \hookrightarrow F$, that is $(FR) \text{ mem} \subset \text{mem } R$, and it is the *largest* one of that type — so a membership relation, if it exists, is unique.

6.5 ✓ The result the section is for: $\text{in}^\circ \text{mem}(F)$ is inductive. It is the immediate-subterm relation.

Nat , $FX = 1 + X$: $\text{mem} = [\emptyset, 1]$ and $[\text{zero}, \text{succ}]^\circ [\emptyset, 1] = \text{succ}^\circ$ is inductive; $< = \text{pred}^+$ with $\text{pred} = \text{succ}^\circ$, which is what made §6.1's recursion terminate.

Lists: $\text{mem} = [\emptyset, \pi']$ and $[\text{nil}, \text{cons}]^\circ[\emptyset, \pi'] = \text{cons}^\circ \pi' = \text{tail}$ is inductive, so proper suffix tail^+ is; on snoc-lists init and proper prefix are.

Unique solutions

- $\text{T } 6.3 \checkmark$ $S \text{ mem}(F) \text{ inductive} \implies X = S(FX)R$ has a unique solution $\varphi(R, S)$, and $\varphi(R, S)$ is entire if R and S are. With $S := S^\circ$ this is the hylomorphism of §6.3, now pinned down as the *only* fixed point.
- $\text{C } 6.2 \checkmark$ $g \text{ mem}(F) \text{ inductive} \implies$ the unique solution of $X = g(FX)f$ is a map ($X = \langle g^\circ \rangle \langle f \rangle$) is entire by the theorem and simple by Ex 6.10).
- $\text{C } 6.3 \checkmark$ $R^\circ \text{ mem}(F) \text{ inductive} \implies \langle R \rangle$ is surjective when R is (R surjective $\triangleq 1 \subset R^\circ R$, i.e. $\text{ran } R = 1$).
- $\text{T } 6.4 \checkmark$ R surjective and $Rf \subset (Ff)$ in $\implies f^\circ = \langle R \rangle$. This is the theorem §5.6 borrowed to show $\text{partition} = \text{concat}^\circ$ is a catamorphism.
- $\text{Ex } 6.12 \checkmark$ $R \text{ inductive} \iff X = X/R$ has a unique solution.
- $\text{Ex } 6.17$ $\text{inlist} : \text{list}^+ \alpha \rightarrow \alpha$ is a catamorphism, but inlist on list α is not; instead $\text{inlist} = \text{tail}^* \text{head}$. The same trick defines intree .
- $\text{Ex } 6.18$ The formal definition: mem is a membership relation for F if $(1/\text{mem})(FR) = R/\text{mem}$ for all R ; equivalently $(S/\text{mem})(FR) = (SR)/\text{mem}$ for all R, S .
- $\text{Ex } 6.20$ $\text{mem}(G)/\text{mem}(F)$ is the largest lax natural transformation $F \hookrightarrow G$.
- $\text{Ex } 6.21$ In a Boolean allegory, $\text{mem}(F)$ is entire $\iff F\emptyset = \emptyset$.

6.6 Sorting by selection

- $(6.6) \checkmark$ $\text{sort} \subset \text{perm}(\text{ordered})$, for R a *connected* preorder ($R \cup R^\circ = \Pi$). A connected preorder need not be anti-symmetric, so sort is genuinely a refinement, not an equality.
- $\text{ordered} = \langle \text{nil}, \text{ok cons} \rangle$, where the coreflexive ok holds at (a, x) when $(\forall b : b \text{ inlist } x : aRb)$ — rebuild the list, checking at each step that only R -smaller elements go in front. ($\text{ok}(a, x) = (x = [] \vee aR(\text{head } x))$) also works but is less useful here.)

Selection sort

$$\begin{aligned} & \text{perm}(\text{ordered}) \\ = & \text{perm} = \text{perm}^\circ \text{ and } \text{ordered} = \text{ordered}^\circ \\ & (\text{ordered perm})^\circ \\ = & \text{ordered} = \langle \text{nil}, \text{ok cons} \rangle \\ & (\langle \text{nil}, \text{ok cons} \rangle \text{ perm}) \\ \supseteq & \text{fusion, for a suitable select} \\ & \langle \text{nil}, \text{select}^\circ \rangle \end{aligned}$$

- 6.6 The fusion proviso is $\text{ok cons perm} \supset (1 \times \text{perm})\text{select}^\circ$, and select is found by calculating:

$$\begin{aligned} & \text{ok cons perm} \\ = & \text{perm} = \langle \text{nil}, \text{cons perm} \rangle \text{ (§5.6)} \\ & \text{ok } (1 \times \text{perm}) \text{ cons perm} \\ = & (1 \times \text{perm})\text{ok} = \text{ok}(1 \times \text{perm}), \text{ Ex } 6.22 \\ & (1 \times \text{perm}) \text{ ok cons perm} \\ \supseteq & \text{specifying } \text{select} \subseteq \text{perm cons}^\circ \text{ ok} \\ & (1 \times \text{perm})\text{select} \end{aligned}$$

- 6.6 In words: $(a, y) = \text{select } x$ means $[a] \# y$ is a permutation of x with aRb for every b in y . It is undefined on $[]$, so take $\text{select} = \text{embed} \langle \text{base}, \text{step} \rangle$ with $\text{embed} : \text{list } \alpha \rightarrow \text{list}^+ \alpha$.

Fusion conditions: $\text{base} \subset \text{wrap perm cons}^\circ \text{ ok}$ and $(1 \times (\text{cons}^\circ \text{ ok})) \text{ step} \subset \text{cons perm cons}^\circ \text{ ok}$, satisfied by

$\text{base } a = (a, [])$ and $\text{step}(a, (b, x)) = (a, [b] \# x)$ if aRb , else $(b, [a] \# x)$.

- $6.6 \checkmark$ The hylomorphism theorem then makes $X = \langle \text{nil}, \text{select}^\circ \rangle$ the unique solution of $X = (\text{nil}^\circ \text{nil}) \cup (\text{select } (1 \times X) \text{ cons})$, i.e. $\text{sort } x = []$ if $x = []$, else $[a] \# \text{sort } y$ where $(a, y) = \text{select } x$.

Quicksort

- 6.6 Same shape, with a tree as the intermediate datatype: $\text{tree } \alpha ::= \text{null} \mid \text{fork}(\text{tree } \alpha, \alpha, \text{tree } \alpha)$ and $\text{flatten} = \langle \text{nil}, \text{join} \rangle$, $\text{join}(x, a, y) = x \# [a] \# y$.

$$\begin{aligned} & \text{perm}(\text{ordered}) \\ \supseteq & \text{flatten a map, so } \text{flatten}^\circ \text{ flatten} \subset 1 \\ & \text{perm flatten}^\circ \text{ flatten ordered} \\ = & \text{claim: } \text{flatten ordered} = \text{inordered flatten} \\ & \text{perm flatten}^\circ \text{ inorder flatten} \\ = & \text{converses} \\ & (\text{inordered flatten perm})^\circ \text{ flatten} \\ \supseteq & \text{fusion, for a suitable split} \\ & \langle \text{nil}, \text{split}^\circ \rangle \text{ flatten} \end{aligned}$$

- 6.6 $\text{inordered} = \langle \text{null}, \text{check fork} \rangle$, with check holding at (x, a, y) when $(\forall b : b \text{ intree } x \implies bRa)$ and $(\forall b : b \text{ intree } y \implies aRb)$; check' is the same with inlist for intree . Writing $Ff = f \times 1 \times f$, the proviso is $F(\text{flatten perm})\text{split}^\circ \subset \text{check fork flatten perm}$.

Discharged by taking $\text{split} \subset \text{perm join}^\circ \text{check}'$, i.e. $(y, a, z) = \text{split } x$ means $y \# [a] \# z$ is a permutation of x with bRa throughout y and aRb throughout z .

$\text{split} = \text{embed} \langle \text{base}, \text{step} \rangle$ with $\text{base} \subset \text{wrap perm join}^\circ \text{check}'$ and $(1 \times (\text{join}^\circ \text{check}')) \text{ step} \subset \text{cons perm join}^\circ \text{check}'$ (the book prints split for step here, and drops the $^\circ$ on join), satisfied by $\text{base } a = ([], a, [])$ and $\text{step}(a, (x, b, y)) = ([a] \# x, b, y)$ if aRb , else $(x, b, [a] \# y)$.

- 6.6 $X = \langle \text{nil}, \text{split}^\circ \rangle \text{ flatten}$ is the least solution of $X = (\text{nil}^\circ \text{nil}) \cup (\text{split}(X \times 1 \times X)\text{join})$, i.e. $\text{sort } x = []$ if $x = []$, else $\text{sort } y \# [a] \# \text{sort } z$ where $(y, a, z) = \text{split } x$.

- $\text{Ex } 6.22$ $\text{inlist} = \text{perm inlist}$; hence a point-free ok and $(1 \times \text{perm})\text{ok} = \text{ok}(1 \times \text{perm})$.

- $\text{Ex } 6.24$ $\text{ordered}(R) \text{ ordered}(S) = \text{ordered}(R \cap S)$.

- $\text{Ex } 6.25$ Sorting a *set*: $\text{sort} \subset \text{setify}^\circ \text{ordered}(R)$ is the wrong specification, $\text{sort} \subset \text{setify}^\circ \text{ordered}(R \cap \text{neq})$ the right one, where $a \text{ neq } b$ iff $a \neq b$.

- $\text{Ex } 6.30$ From $\text{perm} = \langle \text{nil}, \text{add} \rangle$ instead, fusion gives $\text{perm}(\text{ordered}) = \langle \text{nil}, \text{add ordered} \rangle \supset \langle \text{nil}, \text{insert} \rangle$ for a suitable map insert with $(1 \times \text{ordered})\text{insert} \subset \text{add ordered}$ — insertion sort.

6.7 Closure

- $6.7 \checkmark$ R^* , the **REFLEXIVE TRANSITIVE CLOSURE**, is the smallest preorder containing R :

$$R \subset X \iff R^* \subset X \quad \text{for every preorder } X.$$

- $(6.7) \checkmark$ $R^* = (\mu X : 1 \cup (RX))$.

- $(6.8) \checkmark$ $R^* = (\mu X : 1 \cup (XR))$. That $S = (\mu X : 1 \cup (RX))$ is reflexive and contains R is immediate; transitivity goes through division, the same move as Theorem 6.2.

- $\text{Ex } 6.32 \checkmark$ $SR^* = (\mu X : S \cup (XR))$, $RS^* = (\mu X : R \cup (XS))$, $S^*R = (\mu X : R \cup (SX))$.

- $\text{Ex } 6.33$ $R^* = \langle [1, 1] \rangle^\circ \langle [1, R] \rangle$, the intermediate datatype being $\text{iterate } \alpha ::= \text{once } \alpha \mid \text{again}(\text{iterate } \alpha)$.

- $\text{Ex } 6.34$ $R^* = \text{chainR}^\circ \text{head}$ for a catamorphism chainR on non-empty lists.

Ex 6.36 $(R \cup S)^* = (R^* S)^* R^*$, by the diagonal rule.

Ex 6.37 $CR = C \implies R^* = (\sim C R)^*$, for C coreflexive.

When the recursion has a unique solution

6.7 $X = 1 \cup (RX)$ has a unique solution — necessarily R^* — exactly when R is inductive. tail is inductive, so $\text{suffix} = 1 \cup (\text{tail suffix})$ pins suffix down, and transposing gives $\Lambda \text{ suffix} = \langle \tau, \Lambda(\text{tail suffix}) \rangle \text{ cup}$.

tail is undefined on $[]$, so a case analysis splits it:

$(\Lambda \text{ suffix})[] = \{[]\}$ and $(\Lambda \text{ suffix})([a] \# x) = \{[a] \# x\} \cup (\Lambda \text{ suffix})x$ — sets as lists, and this is §5.6's $\text{tails } [] = [[]]$, $\text{tails}([a] \# x) = [[a] \# x] \# \text{tails } x$.

Computing a closure that is not a unique fixed point

6.7 **SUBTRACTION** (Ex 4.30) $R - S \subset T \iff R \subset S \cup T$; hence $R - \emptyset = R$, $R \cup S = R \cup (S - R)$, $R - (S \cup T) = R - S - T$ and $(R \cup S) - T = (R - T) \cup (S - T)$.

Ex 6.35 ✓ **M-CALCULUS** the two defining rules are $\varphi(\mu X : \varphi X) = (\mu X : \varphi X)$ and $\varphi Y \subset Y \implies (\mu X : \varphi X) \subset Y$. From them: *rolling* $(\mu X : \varphi(\psi X)) = \varphi(\mu X : \psi(\varphi X))$, *diagonal* $(\mu X : \mu Y : \varphi(X, Y)) = (\mu X : \varphi(X, X))$, *substitution* $(\mu X : \varphi(\mu Y : \psi(X, Y))) = (\mu X : \psi(\varphi X, X))$.

(6.9) $\theta(P, Q) \triangleq P \cup (\mu X : Q \cup ((XR) - P))$, which satisfies

$$\begin{aligned} \theta(\emptyset, S) &= SR^*, \quad \theta(P, \emptyset) = P, \\ \theta(P, Q) &= \theta(P \cup Q, (QR) - P - Q). \end{aligned}$$

The third comes out of the rolling rule and the subtraction laws.

Read as a program: compute $\theta(\emptyset, S)$, where $\theta(P, Q) = P$ if $Q = \emptyset$, else $\theta(P \cup Q, (QR) - P - Q)$. It terminates only if SR^* is finite; then P grows at every step.

Relations as data are awkward, so apply Λ throughout: with $p = \Lambda P$, $q = \Lambda Q$, $s = \Lambda S$ and $\Lambda(SR^*) = (\Lambda S)(ER^*)$,

$$\begin{aligned} \text{close}(p, \emptyset) &= p, \\ \text{close}(p, q) &= \text{close}(p \cup q, (ER)q - p - q), \end{aligned}$$

now with set union and set difference. This is how $E(R^*)$ is computed whenever the answer is known to be finite — reachability in a graph.

7 Optimisation Problems

7.1 Minimum and maximum

7.1 $\min R \triangleq \exists \cap (\exists^\circ \setminus R) : [\alpha] \rightarrow \alpha$ — a is a minimum of x under R when a is *in* x and is a lower bound of x . R need not be a preorder for the definition to make sense, only for it to be useful.

UNIVERSAL PROPERTY $X \subset \min R \iff X \subset \exists \wedge \exists^\circ X \subset R$.

$\max R = \min R^\circ$.

Three consequences of $f(R/S) = (f^\circ S)/R$, all about dividing by $\exists$:

(7.1) ✓ $\tau(\exists^\circ \setminus R) = R$.

(7.2) $\Lambda(S)(\exists^\circ \setminus R) = S^\circ \setminus R$.

(7.3) ✓ $\bigcup(\exists^\circ \setminus R) = \exists^\circ \setminus (\exists^\circ \setminus R)$.

(7.4) ✓ $\tau(\min R) = 1 \cap R$ — so R is reflexive exactly when the minimum of a singleton is its inhabitant.

(7.5) ✓ $\Lambda(S)(\min R) = S \cap (S^\circ \setminus R)$, whose universal property

$$X \subset \Lambda(S)(\min R) \iff X \subset S \wedge S^\circ X \subset R$$

is the workhorse of the chapter; calculations cite it as *universal property of min*.

(7.6) ✓ **CONTEXT** $\Lambda(S)(\min R) = \Lambda(S)(\min(R \cap S^\circ S))$ — for the purpose of minimising over the sets ΛS returns, R only has to be constrained on values related to one and the same element by S .

Fusion with the power functor

(7.7) $(ES)(\min R) = (\exists S) \cap ((\exists S)^\circ \setminus R)$, from (7.5) and $ES = \Lambda(\exists S)$.

(7.8) Shunting a map through a minimum:
 $(Pf)(\min R) = (\min(fRf^\circ))f$.

(7.9) R reflexive: $(PS)(\min R) = (\exists S) \cap (\exists^\circ \setminus (SR))$. One direction needs tabulations.

(7.10) ✓ **FUSION WITH THE POWER FUNCTOR**
 $(PS)(\min R) \subset (\exists S) \cap (\exists^\circ \setminus (SR))$, always — this half is all that is used later, and it follows in two lines from the naturality of $\exists$.

Distribution over union

(7.11) ✓ R a preorder: $(P(\min R))(\min R) \subset \bigcup(\min R)$ — take a minimum in each set and then a minimum of those. Only an inclusion, because a set in the collection may be empty.

(7.12) $(P(\min R))(\min R) = (P(\text{dom}(\min R))) \bigcup (\min R)$, using $\min R = (\text{dom}(\min R))(\min R)$.

Implementing min

7.1 $\min R$ refines to a map only on non-empty finite sets, and only for R a connected preorder. With $\text{setify} : \text{list}^+ \alpha \rightarrow [\alpha]$ the specification is $\text{minlist } R \subset \text{setify}(\min R)$, and

$\text{minlist } R = \langle 1, \text{bmin } R \rangle$ with $\text{bmin } R(a, b) = (aRb \rightarrow a, b)$ — the *leftmost* minimum, in the case of ties.

Exercises

Ex 7.1 ✓ $\text{subset}^\circ(\exists^\circ \setminus R) = \exists^\circ \setminus R$, where $\text{subset} = \exists/\exists$.

Ex 7.2 ✓ $(ER)\text{subset} = (\exists R)/\exists$.

Ex 7.4 $(PS)(\exists^\circ \setminus R) = \exists^\circ \setminus (SR)$.

Ex 7.5 ✓ R a preorder: $(\exists^\circ \setminus R)R = \exists^\circ \setminus R$.

Ex 7.6 ✓ $\min(R \cap S) = (\min R) \cap (\min S)$.

Ex 7.7 ✓	$\exists^\circ \exists = \Pi$ — every element is in some set; hence $\min R = \exists \iff R = \Pi$.
Ex 7.8	R, S reflexive: $R \cap S^\circ = (\min S)^\circ (\min R)$.
Ex 7.9	R reflexive: $R = \exists^\circ (\min R)$.
Ex 7.10 ✓	R, S reflexive: $\min R \subset \min S \iff R \subset S$.
Ex 7.11 ✓	$\min R$ is simple $\iff R$ is anti-symmetric.
Ex 7.12	R a preorder: $\min R = \exists \cap ((\min R)R)$.
Ex 7.13	R a preorder and S a map: $R \cap (SS^\circ)$ is a preorder.
Ex 7.14 ✓	R a preorder: $\Lambda(R)(\max R) = R \cap R^\circ$.
Ex 7.15	$\langle S(\min R), T(\min R) \rangle \subset \Lambda(S, T)(\min(R \times R))$.
Ex 7.16	R reflexive, S a preorder: $\Lambda(\min S)(\min R) = \min(S; R)$, with $S; R = S \cap (S^\circ \Rightarrow R)$.
Ex 7.19 ✓	MINIMAL ELEMENTS $\text{mnl } R \triangleq \min(R^\circ \Rightarrow R)$. ($R^\circ \Rightarrow R$) is reflexive for every R , but need not be a preorder even when R is.
Ex 7.20 ✓	$\min R \subset \text{mnl } R$, with equality exactly when R is a connected preorder.
Ex 7.22	$(Pf)(\text{mnl } R) = (\text{mnl}(fRf^\circ))f$ — Ex 7.8 for mnl .
Ex 7.23	$\text{mnl } R = \exists \iff R$ symmetric.
Ex 7.25	class $Q = \langle 1, \exists \Lambda(Q) \rangle$ cap picks an equivalence class of Q ; for R a preorder, $\text{mnl}(R; S) = \Lambda(\text{mnl } R) \text{ class}(R \cap R^\circ) \text{ mnl } S$.
Ex 7.26	R WELL BOUNDED $\triangleq \text{dom } \exists = \text{dom}(\min R)$ — every non-empty set has a minimum. A well-bounded preorder is necessarily connected, and R is well bounded $\iff$ its strict part $R \cap \neg R^\circ$ is well-founded.
Ex 7.27	R well bounded $\implies fRf^\circ$ well bounded, for any map f .
Ex 7.28	R well bounded $\iff \exists \subset (\min R)R^\circ$.
Ex 7.29	R a well-bounded preorder: $\bigcup(\min R) = (E(\min R))(\min R)$ — (7.11) strengthened to an equality, which is what makes $\text{setify}(\min R)$ a catamorphism.
Ex 7.30	R WELL SUPPORTED $\triangleq \text{dom } \exists = \text{dom}(\text{mnl } R)$ — weaker than well bounded.
Ex 7.31	R a well-supported preorder: $\exists \subset (\text{mnl } R)R^\circ$.
Ex 7.32	R a well-supported preorder: $\bigcup(\text{mnl } R) = (E(\text{mnl } R))(\text{mnl } R)$.

7.2 Monotonic algebras

7.2 ✓	An F -algebra $S : F\alpha \rightarrow \alpha$ is MONOTONIC ON $R : \alpha \rightarrow \alpha$ if $(FR)S \subset SR.$ Addition is monotonic on $\leq$: $(\text{leq} \times \text{leq})\text{plus} \subset \text{plus leq}$, i.e. $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$. For a map: f monotonic on $R \iff f^\circ(FR)f \subset R \iff FR \subset fRf^\circ$; by shunting, f is monotonic on $R \iff$ on R°. Neither equivalence survives for a general S. f DISTRIBUTES OVER R if $F(\min R)f \subset \Lambda((F\exists)f)(\min R)$ — pointwise, for $+$ over $\leq$, $\min x + \min y = \min\{a + b \mid a \in x \wedge b \in y\}$ on non-empty x, y.
T 7.1 ✓	A map f is monotonic over $R \iff$ it distributes over R .

Theorem (Theorem 7.2 — the greedy theorem ✓).

S monotonic on the preorder R° :

$$(\Lambda(S)(\min R)) \subset \Lambda(\Lambda(S))(\min R).$$

Take the locally best step at every stage and the result is globally optimal. The proof is the universal property of $\min$, then the hylomorphism theorem: the greedy fold is below every prefixed point of the associated recursion, and transitivity of R discharges it. For $\max$ the condition is monotonicity on R , not R° ; and context can always be brought in, so monotonicity on $R^\circ \cap (\Lambda(S)^\circ \Lambda(S))$ suffices.

Ex 7.33 ✓	$a + b \leq a + b' \implies b \leq b'$, point-free.
Ex 7.34 ✓	in monotonic on $R \implies R$ is reflexive.
Ex 7.37 ✓	A variant: f monotonic on R and $f \subset \Lambda(S)(\min R) \implies \Lambda(f) \subset \Lambda(\Lambda(S))(\min R)$.
Ex 7.38 ✓	S monotonic on $R^\circ \implies (\min(FR))\Lambda(S)(\min R) \subset (ES)(\min R)$ — optimal substeps.
Ex 7.39	$\text{takewhile } p = \Lambda((\text{list } p)(\text{prefix}))(\max R)$ with $R = \text{length leq length}$; with $\text{prefix} = \langle \text{nil}, \text{cons} \cup \text{nil} \rangle$ the greedy theorem gives the standard program.
Ex 7.40	MAXIMUM SEGMENT SUM $\text{mss} = \Lambda(\text{segment sum})(\max)$, and with $\text{segment} = \text{suffix prefix}$, $\text{mss} = \Lambda \text{ suffix } P(\Lambda(\text{prefix sum})(\max))(\max)$; the greedy theorem gives $\langle \text{zero}, \text{oplus} \rangle \subset \Lambda(\text{prefix sum})(\max)$ with $\text{oplus} = \Lambda(\text{zero} \cup \text{plus})(\max)$, and a linear-time program follows.
Ex 7.41	filter $p = \Lambda((\text{list } p)(\text{subseq}))(\max R)$; with $\text{subseq} = \langle \text{nil}, \text{cons} \cup \pi' \rangle$ the greedy theorem gives the standard program.
Ex 7.42	L the lexical order: $(1 \times L)\text{cons} \subset \text{cons } L$, hence $\langle \text{nil}, \Lambda(\text{cons} \cup \pi')(\max L) \rangle \subset \Lambda(\text{subseq})(\max L)$; the greedy step is $(\text{ok} \rightarrow \text{cons}, \pi')$ with $\text{ok}(a, x)$ iff $x = []$ or $a \geq \text{head } x$.

7.3 Planning a company party

7.3 ✓	A company is a tree $\alpha ::= \text{node}(\alpha, \text{list}(\text{tree } \alpha))$; each employee has a <i>conviviality</i> rating, and no employee may attend with their immediate supervisor. Maximise the total rating of the guest list. Base functor $F(A, B) = A \times \text{list } B$. $\Lambda \text{ party}(\max R)$ with $R = (\text{list rating sum}) \text{ leq } (\text{list rating sum})^\circ$ and $\text{party} = \langle \text{include}, \text{exclude} \rangle \text{ choose}$, $\text{choose} = \pi \cup \pi'$ — one catamorphism producing the two parties, one with the root and one without. $\text{include} = (1 \times (\text{list } \pi' \text{ concat}))\text{cons}$ (a map: take the root, so the roots of the immediate subtrees are excluded), $\text{exclude} = (1 \times (\text{list choose concat}))\pi'$ (not a map: each immediate subtree is free to include or exclude its root). $\Lambda \text{ party}(\max R)$ $=$ definition of party $\Lambda(\langle \text{include}, \text{exclude} \rangle \text{ choose})(\max R)$ $=$ $\Lambda(XY) = \Lambda(X)(EY)$ $\Lambda \langle \text{include}, \text{exclude} \rangle (E \text{ choose})(\max R)$ $\supseteq$ Ex 7.38, since choose is monotonic on R $\Lambda \langle \text{include}, \text{exclude} \rangle (\max(R \times R)) \Lambda \text{ choose}(\max R)$ $\supseteq$ greedy, since $\langle \text{include}, \text{exclude} \rangle$ is monotonic on $R \times R$ $\langle \Lambda \langle \text{include}, \text{exclude} \rangle (\max(R \times R)) \rangle \Lambda \text{ choose}(\max R)$
-------	---

7.3 ✓	The two conditions are $(R \times R)\text{choose} \subset \text{choose } R$ and $(1 \times \text{list}(R \times R))\langle \text{include}, \text{exclude} \rangle \subset$
-------	--

$\langle \text{include}, \text{exclude} \rangle (R \times R)$, the second reducing by Ex 7.15 to $\Lambda \text{ include}(\max R)$ and $\Lambda \text{ exclude}(\max R)$ separately.

include is already a map, and exclude refines to $\pi'(\text{list}(\Lambda \text{ choose}(\max R)))\text{concat}$. The result: a linear-time greedy sweep for a problem posed as an exercise in dynamic programming.

7.4 Shortest paths on a cylinder

7.4 ✓ An $n \times m$ array of positive integers rolled into a cylinder, the top and bottom rows adjacent. A path enters at the left, and from a square may step to one of the three adjacent squares of the next column. Find a path of least total cost, in $O(nm)$.

N is the type functor of n -tuples, $L = \text{list}^+$ with base functor $F(A, X) = A + (A \times X)$. The problem is $\Lambda \text{ paths}(\min R)$ with $R = \text{sum leq sum}^\circ$ and $\text{paths} : L \ N \ \text{Nat} \rightarrow [\alpha]$.

paths is not the transpose of a catamorphism, so it is built from one: $\text{paths} = \langle \text{generate} \rangle \text{setify} \bigcup$ with $\text{generate} : F(N \ \alpha, N \ P \ L \ \alpha) \rightarrow N \ P \ L \ \alpha$, a *lax natural transformation*, assembled by type from $\text{generate} = F(1, \text{moves trans } N \bigcup) \text{zip cp } N(P \text{ in})$.

$\text{moves } x = \{\text{up } x, x, \text{down } x\}$ rotates columns, trans transposes a set of tuples, zip commutes N with F , $\text{cp} = \Lambda(F(1, \emptyset))$.

(7.13) ✓ The crux is Theorem 7.1, not the greedy theorem: in is a monotonic map, hence distributes, so

$$F(1, \min R) \text{ in } \subset \text{cp } (P \text{ in}) (\min R).$$

$\Lambda \text{ paths } (\min R)$
 $=$ definition of paths
 $\Lambda \langle \text{generate} \rangle \text{setify} \bigcup (\min R)$
 $\supseteq$ distribution over union (7.11), R a preorder
 $\Lambda \langle \text{generate} \rangle \text{setify } (P(\min R))(\min R)$
 $\supseteq$ naturality of setify
 $\Lambda \langle \text{generate} \rangle (N(\min R)) \text{setify } (\min R)$
 $\supseteq$ fusion, for the Q below
 $\langle Q \rangle \text{setify } (\min R)$

7.4 ✓ The fusion condition is $\text{generate } N(\min R) \supset F(1, N(\min R))Q$, and unfolding it through (7.13) and the naturalities of zip , trans and moves gives $Q = \text{moves trans } (N \ P(\min R))(N(\min R)) F(1, \cdot) \text{zip } N \text{ in}$, which as a coproduct is $Q = [N \ \text{wrap}, (1 \times (\text{moves trans } N(\min R)))\text{zip}' N \ \text{cons}]$.

Sets and n -tuples both become lists, $\min R$ becomes $\text{minlist } R$, and zip the standard function — a single left-to-right sweep, one column at a time.

7.5 The security van problem

7.5 ✓ A bank's cash must stay between 0 and N ; a security van tops it up or takes a surplus. Given a sequence of transactions, cut it into as few *secure* segments as possible. The moral of the section: when the monotonicity condition fails, refine the *ordering* instead of abandoning the greedy algorithm.

$\text{ceiling} = \Lambda(\text{prefix sum})(\max \text{leq})$ and $\text{floor} = \Lambda(\text{prefix sum})(\min \text{leq})$; x is secure iff $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$. secure is the corresponding coreflexive.

The problem is $\Lambda(\text{partition list secure})(\min R)$ with $R = \text{length leq length}^\circ$; fusing list secure into $\text{partition} = \langle \text{nil}, \text{new } \cup \text{glue} \rangle$ gives

$$\text{partition list secure} = \langle \text{nil}, \text{new } \cup \text{old} \rangle,$$

$$\text{old} = (1 \times \text{cons}^\circ) \text{assocl}((\text{cons secure}) \times 1) \text{cons}.$$

Greedy needs $[\text{nil}, \text{new } \cup \text{old}]$ monotonic on R° , i.e.

$$(7.14) \quad (1 \times R^\circ) \text{new} \subset (\text{new } \cup \text{old}) R^\circ \text{ — true.}$$

$$(7.15) \quad (1 \times R^\circ) \text{old} \subset (\text{new } \cup \text{old}) R^\circ \text{ — false: two partitions of the same sequence may have equal length without } \text{secure}([a] \# x) \text{ implying } \text{secure}([a] \# y).$$

secure is *prefix-closed* — $\text{secure prefix} \subset \text{prefix secure}$ — so refine R to $R; H$, where $H = (\text{head prefix head}^\circ) \cup (\text{nil}^\circ \text{ nil})$ and $R; H = R \cap (R^\circ \Rightarrow H)$: same length, and the first component a prefix.

With the refined order, (7.16) $(1 \times (R; H)^\circ) \text{new} \subset (R; H)^\circ (\text{new } \cup \text{old})$ follows from (7.18) $(1 \times \Pi) \text{new} \subset \text{new } H^\circ$, and (7.17) for old from the three claims (7.19)–(7.21) about the strict part $|R| = R \cap \neg R^\circ$. Hence

$$\langle (\text{ok} \rightarrow \text{glue}, \text{new}) (\text{wrap wrap}) \rangle \subset \langle \Lambda(S)(\min(R; H)) \rangle,$$

with ok holding at (a, xs) when $xs \neq []$ and $[a] \# \text{head } xs$ is secure.

secure turns out to be suffix-closed too, which is what makes the test efficient: $\text{ceiling} = \langle \text{zero}, \text{omax plus} \rangle$ and $\text{floor} = \langle \text{zero}, \text{omin plus} \rangle$ (two baby applications of the greedy theorem), so $\text{ok } N(a, [x] \# xs) = (\text{omax}(a + \text{ceiling } x) - \text{omin}(a + \text{floor } x)) \leq N$.

Ex 7.57 The refined order can be given directly: $Q = (\text{nil}^\circ !) \cup (\text{prefix}; (\text{tail}^\circ Q \text{ tail}))$, a preorder and a linear order on partitions of one sequence. $Q \not\subset R$, yet $\Lambda(\langle S \rangle)(\min Q) \subset \Lambda(\langle S \rangle)(\min R)$ whenever secure is suffix-closed.

8 Thinning Algorithms

Between the greedy theorem's single partial solution and Eilenberg–Wright's exhaustive set, keep a *representative* collection: those partial solutions that might still extend to an optimum.

8.1 Thinning

- (8.1) ✓ **THIN** $\text{thin } Q = (\exists/\exists) \cap (\exists^\circ \setminus (Q\exists^\circ)) : [\alpha] \rightarrow [\alpha]$ — take a set y to a subset x of it such that every member of y has a Q -lower bound in x . Nothing is lost that a Q -minimum could have used. Unlike $\min R$, Q need not be a connected preorder for $\text{thin } Q$ to be implementable (§8.3).
- ✓ **UNIVERSAL PROPERTY** $X \subset \Lambda(S)(\text{thin } Q) \iff X\exists \subset S \wedge S^\circ X \subset Q\exists^\circ$ — as usual the workhorse; every proof below is this property applied twice.
- ✓ $Q \subset R \implies \text{thin } Q \subset \text{thin } R$; and $\text{thin } Q$ is reflexive, transitive, a preorder exactly when Q is. *From here on Q is a preorder.*
- ✓ **THIN-INTRODUCTION** $\min R = (\text{thin } Q)(\min R)$ when $Q \subset R$ — thinning first changes no minimum.
- (8.2) ✓ **THIN-ELIMINATION** $\text{thin } Q \supset (\min Q)\tau$, where τ wraps a value in a singleton. So thinning can always be refined to “take the minimum”. Two cautions: unless Q is connected, $(\min Q)\tau$ has the smaller domain — $\text{thin } 1$ is entire while $(\min 1)\tau$ is defined only on singletons, so the refinement can be infeasible. At the other end $\text{thin } Q \supset 1$, so it can always be refined away entirely.
- (8.3) ✓ The usable form: $\Lambda(S)(\text{thin } Q) \supset \Lambda(S)(\min R)\tau$ provided $R \cap (S^\circ S) \subset Q$.
- $$\begin{aligned} & S^\circ \Lambda(S)(\min R)\tau \subset Q\exists^\circ \\ \equiv & \text{shunt } \tau; \tau\exists = 1 \\ & S^\circ \Lambda(S)(\min R) \subset Q \\ \equiv & \text{context, (7.6)} \\ & S^\circ \Lambda(S)(\min(R \cap (S^\circ S))) \subset Q \\ \Leftarrow & S^\circ \Lambda(S) \subset \exists^\circ \\ & \exists^\circ \min(R \cap (S^\circ S)) \subset Q \\ \Leftarrow & \text{definition of min} \\ & R \cap (S^\circ S) \subset Q \end{aligned}$$
- (8.4) ✓ $\bigcup(\text{thin } Q) \supset (P(\text{thin } Q)) \bigcup$ — thinning each set and then uniting refines uniting and then thinning.
- T 8.1 ✓ **THE THINNING THEOREM** S monotonic on Q° .
Then
$$\|\Lambda((F\exists)S)(\text{thin } Q)\| \subset \Lambda(\|S\|)(\text{thin } Q).$$
- Two conditions, by the universal property: $\|\dots\|\exists \subset \|\|S\|\|$, an exercise in fusion, and $\|\|S\|\|^\circ \|\dots\| \subset Q\exists^\circ$, which by the hylomorphism theorem follows from:
- $$\begin{aligned} & S^\circ F(Q\exists^\circ) \Lambda((F\exists)S)(\text{thin } Q) \\ \subset & S^\circ (FQ) \subset QS^\circ, \text{ monotonicity converted} \\ & QS^\circ (F\exists^\circ) \Lambda((F\exists)S)(\text{thin } Q) \\ \subset & X^\circ \Lambda(X) \subset \exists^\circ \\ & Q\exists^\circ (\text{thin } Q) \\ \subset & \exists^\circ (\text{thin } Q) \subset Q\exists^\circ \\ & QQ\exists^\circ \\ = & Q \text{ transitive} \\ & Q\exists^\circ \end{aligned}$$
- C 8.1 ✓ $Q \subset R$ and S monotonic on Q° :
$$\|\Lambda((F\exists)S)(\text{thin } Q)\|(\min R) \subset \Lambda(\|S\|)(\min R)$$
 — thin-introduction applied to Theorem 8.1.

Exhaustive search ($Q = 1$) and the greedy algorithm are the two extremes.

- Ex 8.1 ✓ $\text{thin } 1 = 1$.
- Ex 8.2 ✓ $\text{thin } Q$ is a preorder if Q is.
- Ex 8.3 ✓ $(\text{thin } Q)(\min R) \subset \min R$ when $Q \subset R$ — half of thin-introduction.
- Ex 8.4 ✓ $\langle \text{thin } R, \text{thin } Q \rangle \text{ cup} \subset \text{thin } R$, from the stronger $\langle \text{thin } R, \exists/\exists \rangle \text{ cup} \subset \text{thin } R$, which needs $\exists \times \exists \subset \text{cup } \langle \exists, \exists \rangle$.
- Ex 8.5 ✓ $\min R = (\text{thin } R)\tau^\circ$, whence thin-elimination.
- Ex 8.6 ✓ **CONTEXT** $\Lambda(S)(\text{thin } Q) = \Lambda(S)(\text{thin}(Q \cap (S^\circ S)))$.
- Ex 8.8 ✓ The greedy algorithm is Theorem 8.1 at the other extreme.
- Ex 8.9 ✓ Q well-supported (Ex 7.30) $\implies \Lambda(\text{mnl } Q) \subset \text{thin } Q$ — keeping the minimal elements is a legitimate thinning.

8.2 Paths in a layered network

- 8.2 ✓ A *layered network* is a non-empty sequence of sets of vertices; a *path* picks one vertex from each layer, at cost $\text{cost}[a_0, \dots, a_n] = \sum_{0 \leq j < n} \text{wt}(a_j, a_{j+1})$. Find a least-cost path. Non-empty cons-lists, paths built right to left, $R = \text{cost leq cost}^\circ$:
- $$\text{minpath} \subset \Lambda(\text{list}^+ \exists)(\min R) = \Lambda(\|F(\exists, 1) \text{ in}\|)(\min R),$$
- with $\text{in} = [\text{wrap}, \text{cons}]$ and F the base bifunctor.
- cost is not a catamorphism, but pairs with one:
- $$\begin{aligned} \|\text{wrapz}, \text{consw}\| &= \langle 1, \text{cost} \rangle, \\ \text{wrapz} &= \langle \text{wrap}, \text{zero} \rangle, \\ \text{consw}(a, (x, n)) &= (\text{cons}(a, x), \text{wt}(a, \text{head } x) + n). \end{aligned}$$
- Corollary 8.1 needs $Q \subset R$ with $F(\exists, Q^\circ)$ in $\subset F(\exists, 1)$ in Q° . $Q = R$ fails: for paths p, q with cost $p \geq \text{cost } q$ it would demand $\text{wt}(a, \text{head } p) - \text{wt}(a, \text{head } q) \geq \text{cost } q - \text{cost } p$, and $\text{wt}(a, \text{head } q)$ can be arbitrarily large. The fix is to compare only paths with the same first vertex:
- $$Q = R \cap (\text{head head}^\circ),$$
- $$\text{minpath} \subset \|\Lambda(F(\exists, \exists) \text{ in})(\text{thin } Q)\|(\min R).$$
- Operationally: keep one partial solution per starting vertex, a shortest path beginning there. That is exactly what the thinning step can be eliminated into.

$$\begin{aligned} & \Lambda(F(\exists, \exists) \text{ in})(\text{thin } Q) \\ = & \text{bifunctors} \\ & \Lambda(F(\exists, 1) F(1, \exists) \text{ in})(\text{thin } Q) \\ = & \Lambda \text{ of a composite} \\ & \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists) \text{ in})) \bigcup (\text{thin } Q) \\ \supset & (8.4) \\ & \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists) \text{ in})(\text{thin } Q)) \bigcup \\ \supset & (8.3) \\ & \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists) \text{ in})(\min R)\tau) \bigcup \\ = & (P\tau) \bigcup = 1 \\ & \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists) \text{ in})(\min R)) \\ = & P = E \text{ on maps} \\ & \Lambda(F(\exists, 1)) P(\Lambda(F(1, \exists))(P \text{ in})(\min R)) \end{aligned}$$

(8.3) applies because $R \cap (S^\circ S) \subset Q$: it reduces to $S^\circ S \subset \text{head head}^\circ$, that is S head simple, and $S \text{ head} \subset [1, \pi]$.

Nothing above used the datatype. For non-empty cons-lists, $\Lambda(F(\exists, 1)) = 1 + \text{cpl}$ and $\Lambda(F(1, \exists))(P \text{ in})(\min R) = [\text{wrap}, \text{step}]$ with $\text{step} = \text{cpr } (P \text{ cons})(\min R)$, so finally

$$\text{minpath} \subset \|\Lambda(P \text{ wrap}, \text{cpl } (P \text{ step}))\|(\min R),$$

which represents a path p by $(p, (\text{head } p, \text{cost } p))$.

Ex 8.11 The general law behind the whole section: $Q \subset R$, $S = S_2 S_1$ monotonic on Q° , $R \cap (S_1^\circ S_1) \subset Q$. Then $\llbracket \Lambda((F \ni) S_2) P (\Lambda(S_1)(\min R)) \rrbracket (\min R) \subset \Lambda(\llbracket S \rrbracket)(\min R)$.

8.3 Implementing thin

8.3 ✓ thin Q becomes a program only on finite sets — but, unlike $\min R$, the sets may be empty and Q need not be connected. On lists:

$$(\text{thinlist } Q) \text{setify} \subset \text{setify}(\text{thin } Q),$$

$$\text{thinlist } Q \subset \text{subseq},$$

the second saying the relative order survives.

(8.5) The ideal is linear time and a shortest result; in particular $\text{thinlist } Q x = [\minlist Q x]$ when Q is connected and x non-empty. $\text{thinlist } Q = 1$ is legitimate and useless.

Drop an element when a neighbour bumps it — linear in the number of Q -tests, though not always shortest:

$$\text{thinlist } Q = \llbracket \text{nil}, \text{bump } Q \rrbracket,$$

$$\text{bump } Q(a, []) = [a],$$

$$\text{bump } Q(a, [b] \# x) = \begin{cases} [a] \# x & aQb \\ [b] \# x & bQa \\ [a] \# [b] \# x & \text{else.} \end{cases}$$

Sorting sets

8.3 sort $P = \text{setify}^\circ(\text{ordered } P)$ for a connected preorder P — not a map even for a linear order, since Rel sort $\text{leq } \{1, 2, 3\}$ may give $[1, 2, 3]$ or $[1, 1, 2, 3]$.

(8.6) $(\text{sort } P)(\text{thinlist } Q) \subset (\text{thin } Q)(\text{sort } P)$ — sorting then thinning a list refines thinning the set.

$$\begin{aligned} & (\text{sort } P)(\text{thinlist } Q) \\ = & \text{sort } P \\ & \text{setify}^\circ(\text{ordered } P)(\text{thinlist } Q) \\ & (\text{ordered } P)(\text{thinlist } Q) \subset (\text{thinlist } Q)(\text{ordered } P), \text{ since} \\ & \subset (\text{ordered } P) \text{subseq} \subset \text{ordered } P \text{ for connected } P \\ & \text{setify}^\circ(\text{thinlist } Q)(\text{ordered } P) \\ & \subset \text{specification of thinlist, shunted} \\ & (\text{thin } Q) \text{setify}^\circ(\text{ordered } P) \\ = & \text{sort } P \\ & (\text{thin } Q)(\text{sort } P) \end{aligned}$$

P is not free: it should bring Q -comparable elements together. If Q is connected and $P = Q$, thinning is just “return the first element”.

$$(8.7) \quad (\text{sort } P)(\minlist Q) \subset \min Q.$$

$$(8.8) \quad \text{sort}(f P f^\circ)(\text{list } f) \subset (P f)(\text{sort } P).$$

$$(8.9) \quad (\text{sort } P)(\text{filter } p) \subset (E p)(\text{sort } P), \text{ for } p \text{ a coreflexive.}$$

$$(8.10) \quad (\text{sort } P)^2(\text{merge } P) \subset \text{cup}(\text{sort } P).$$

(8.11) $F(\text{sort } P)(\text{listcp}(F)) \subset \text{cp}(F)(\text{sort}(F P))$, where $\text{cp}(F) = \Lambda(F \ni)$ is §5.6’s cartesian product and $\text{listcp}(F)$ its list version. Not every F admits one: F must distribute over arbitrary joins. A polynomial functor that does is called **LINEAR**, and for those (8.11) can be met — assumed from here on.

L 8.1 f monotonic on P , p a coreflexive. Then $F(\text{sort } P)(\text{listcp}(F))(\text{list } f)(\text{filter } p) \subset \Lambda((F \ni) f p)(\text{sort } P)$.

$$\begin{aligned} & \Lambda((F \ni) f p)(\text{sort } P) \\ = & \Lambda \text{ of a composite; } \text{cp}(F) = \Lambda(F \ni) \\ & \text{cp}(F)(E(f p))(\text{sort } P) \\ = & E \text{ a functor, and } E = P \text{ on maps} \\ & \text{cp}(F)(P f)(E p)(\text{sort } P) \end{aligned}$$

$$\begin{aligned} & (8.9) \\ & \text{cp}(F)(P f)(\text{sort } P)(\text{filter } p) \\ & (8.8) \\ & \text{cp}(F)(\text{sort}(f P f^\circ))(\text{list } f)(\text{filter } p) \\ & f \text{ monotonic on } P \\ & \text{cp}(F)(\text{sort}(F P))(\text{list } f)(\text{filter } p) \\ & (8.11) \\ & F(\text{sort } P)(\text{listcp}(F))(\text{list } f)(\text{filter } p) \end{aligned}$$

Binary thinning

T 8.2 **THE BINARY THINNING THEOREM** Suppose $S = (f_1 p_1) \cup (f_2 p_2)$ with p_1, p_2 coreflexive; Q a preorder, $Q \subset R$, with $f_1 p_1$ and $f_2 p_2$ both monotonic on Q° ; and P a connected preorder with f_1, f_2 monotonic on P . Then, for $g_i = (\text{list } f_i)(\text{filter } p_i)$,

$$\begin{aligned} & \llbracket \text{listcp } \langle g_1, g_2 \rangle (\text{merge } P) (\text{thinlist } Q) \rrbracket (\minlist R) \\ & \subset \Lambda(\llbracket S \rrbracket)(\min R). \end{aligned}$$

$$\begin{aligned} & \Lambda(\llbracket S \rrbracket)(\min R) \\ & \supset \text{Theorem 8.1, } S \text{ monotonic on } Q^\circ \\ & \llbracket \Lambda((F \ni) S) (\text{thin } Q) \rrbracket (\min R) \\ & \supset (8.7) \\ & \llbracket \Lambda((F \ni) S) (\text{thin } Q) \rrbracket (\text{sort } P)(\minlist R) \\ & \supset \text{fusion, by the chain below} \\ & \llbracket \text{listcp } \langle g_1, g_2 \rangle (\text{merge } P) (\text{thinlist } Q) \rrbracket (\minlist R) \end{aligned}$$

$$\begin{aligned} & \Lambda((F \ni) S) (\text{thin } Q) (\text{sort } P) \\ & \supset (8.6) \\ & \Lambda((F \ni) S) (\text{sort } P) (\text{thinlist } Q) \\ = & \text{definition of } S \\ & \langle \Lambda((F \ni) f_1 p_1), \Lambda((F \ni) f_2 p_2) \rangle \text{cup } (\text{sort } P) (\text{thinlist } Q) \\ & \supset (8.10) \\ & \langle \Lambda((F \ni) f_1 p_1) (\text{sort } P), \Lambda((F \ni) f_2 p_2) (\text{sort } P) \rangle \\ & (\text{merge } P) (\text{thinlist } Q) \\ & \supset \text{Lemma 8.1} \\ & F(\text{sort } P) \text{listcp } \langle g_1, g_2 \rangle (\text{merge } P) (\text{thinlist } Q) \end{aligned}$$

Ex 8.12 On snoc-lists $\text{thinlist } Q = \llbracket 1, \text{bump } Q \rrbracket$ — a different function from the cons-list one above.

Ex 8.17 $(\text{sort } P) \text{subseq} \subset (\exists/\exists)(\text{sort } P)$ for connected P .

Ex 8.19 listcp of a product and of a coproduct from the parts; listcp of the identity functor and of a constant functor.

Ex 8.21 (8.11) genuinely needs linearity: there is a polynomial relator that is not linear and for which it fails.

Ex 8.22 The mirror-image theorem, for $S = (p_1 f_1) \cup (p_2 f_2)$.

8.4 The knapsack problem

8.4 ✓ Pack items of given weight and value into a knapsack of capacity w , at greatest total value. Selections are subsequences:

$$\text{subseq} = \llbracket [\text{nil}, \text{cons}] \cup [\text{nil}, \pi'] \rrbracket,$$

$$\text{value} = (\text{list val}) \text{sum},$$

$$\text{weight} = (\text{list wt}) \text{sum},$$

$$\text{knapsack } w \subset \Lambda(\text{subseq}(\text{within } w))(\max R),$$

with $R = \text{value leq value}^\circ$ — or the same with $\min$ and $R = \text{value geq value}^\circ$. Weights are non-negative, so fusion pushes the capacity test inside: $\text{within } w \text{subseq} = \llbracket ([\text{nil}, \text{cons}] (\text{within } w)) \cup [\text{nil}, \pi'] \rrbracket$ — written as a union of two, not collapsed, because that is the shape binary thinning wants.

No greedy algorithm: $[\text{nil}, \text{cons}]$ and $[\text{nil}, \pi']$ are monotonic on R° , but $[\text{nil}, \text{cons}] (\text{within } w)$ is not — a lighter selection of no greater value may still be the one that fits. It *is* monotonic on Q° for

$$Q = R \cap (\text{weight leq weight}^\circ),$$

and so is $[\text{nil}, \pi']$. The cons-list functor is linear, so with $P = R$ — sorting by descending value — the binary thinning theorem applies.

With $FA = 1 + (\text{Item} \times A)$, $\text{listcp} = \text{wrap} + \text{cpr}$, and g_1, g_2 split as coproducts, everything simplifies to
 knapsack $w = (\text{nil}, \text{cpr } \langle h_1, h_2 \rangle (\text{merge } R)(\text{thinlist } Q))$
 $(\text{minlist } R),$

$$h_1 = (\text{list cons})(\text{filter}(\text{within } w)),$$

$$h_2 = \text{list } \pi'.$$

Packings come out in descending value, so $\text{minlist } R$ is just head.

Exponential in the worst case, fast in practice, and — unlike the textbook dynamic programming solution — it does not need the weights and the capacity to be integers. When they are, it is as efficient.

8.5 The paragraph problem

8.5 ✓ Break a sequence of words into lines of width at most w , wasting as little as possible. Line = $\text{list}^+ \text{ Word}$, Para = $\text{list}^+ \text{ Line}$, cons-lists:

$$\text{paragraph } w \subset \Lambda(\text{partition}(\text{list}^+(\text{fits } w)))(\text{min } R),$$

$$R = (\text{waste } w) \text{ leq } (\text{waste } w)^\circ,$$

$$\text{partition} = (\text{wrap wrap, new } \cup \text{ glue}),$$

$$\text{new}(a, xs) = [[a]] \# xs,$$

$$\text{glue}(a, xs) = [[a]] \# \text{head } xs \# \text{tail } xs.$$

glue is a total map on non-empty lists — which the algorithm needs. Width counts one unit per interword space, and waste ignores the last line:

$$\text{width} = (\text{length}, (\text{length} \times 1)\text{plus succ}),$$

$$\text{waste } w = \text{init}(\text{list}(\text{white } w))\text{collect},$$

$$\text{white } wx = w - \text{width } x,$$

$$\text{collect} = (\text{list sqr})\text{sum}.$$

Assuming each word fits on a line by itself, fusion gives paragraph $w \subset \Lambda((\text{wrap wrap, new } \cup \text{ glue}(\text{ok } w)))(\text{min } R)$, and the algebra rewrites as a union of two maps, $[\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}](\text{ok } w))$ — binary thinning again.

Before proceeding with the derivation of an algorithm, we note that the obvious greedy algorithm does not solve this specification.	Before proceeding with the derivation of an algorithm, we note that the obvious greedy algorithm does not solve this specification.
---	---

Figure 8.1: a greedy and an optimal paragraph. The greedy algorithm fills each line as far as it will go; glue is not monotonic on R° , and two partitions of the same input compare only when their first lines agree. So

$Q = R \cap (\text{head head}^\circ)$ — and both new and $\text{glue}(\text{ok } w)$ are monotonic on Q° , given that cons is monotonic under $\text{collect leq collect}^\circ$, which the squared-white-space collect satisfies.

$P = R$ is out, as the monotonicity fails. $P = \text{head}^\circ L \text{ head}$ for a linear order L on lines works — prefix will do, being linear on lines of one input. But the simplest choice suffices: $P = \Pi$. Every map is monotonic on Π , and

$\text{merge } \Pi = \text{cat}$, which already brings together every partial solution with the same first line.

With $FA = \text{Word} + (\text{Word} \times A)$ and $\text{listcp} = \text{wrap} + \text{cpr}$:

$$\text{paragraph } w = (\text{start, cpr } \langle h_1, h_2 \rangle \text{ cat}(\text{thinlist } Q))$$

$$(\text{minlist } R),$$

$$\text{start} = \text{wrap wrap wrap},$$

$$h_1 = \text{list new},$$

$$h_2 = (\text{list glue})(\text{filter}(\text{ok } w)).$$

A partition $[x] \# xs$ is carried as $([x] \# xs, (w - \text{width } x, \text{waste } w xs))$.

8.6 Bitonic tours

8.6 ✓ A *bitonic* tour starts at the leftmost city, goes strictly left to right to the rightmost, and returns strictly right to left. The euclidean travelling salesman problem is NP-complete; its bitonic restriction is quadratic. Generalised here: costs $\text{tc}(a, b)$ need be neither euclidean, nor positive, nor symmetric.

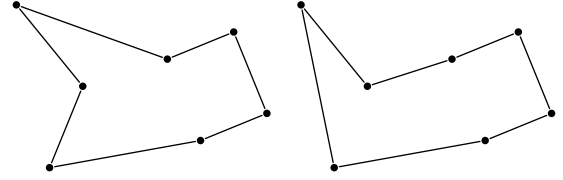


Figure 8.2: an optimal and an optimal bitonic tour of the same seven points. On the right the outward journey runs left to right along the upper chain and the return comes back along the lower one.

A tour is a *pair* of lists (x, y) , the outward journey and the return, read right to left; both are subsequences of the input of length at least two, sharing their first and last elements. $\text{Rel } ([\text{New York, Rome}], [\text{New York, London, Rome}])$ visits London on the way back, and is a different tour from the pair the other way round, because costs may depend on direction. Base functor $FA = (\text{City} \times \text{City}) + (\text{City} \times A)$ and

$$\text{tour} = (\text{start, dropl } \cup \text{ dropr}),$$

$$\text{start}(a, b) = ([a, b], [a, b]),$$

$$\text{dropl}(a, ([b] \# x, y)) = ([a] \# x, [a] \# y),$$

$$\text{dropr}(a, (x, [b] \# y)) = ([a] \# x, [a] \# y),$$

the two ways of not visiting the dropped city twice. $\text{cost}(x, y) = \text{outcost } x + \text{incost } y$, and mintour refines $\Lambda(\text{tour})(\text{min } R)$ with $R = \text{cost leq cost}^\circ$.

Monotonicity on R° needs $(1 \times R^\circ)\text{dropl} \subset \text{dropl } R^\circ$ and the same for dropr . Expanding $\text{cost}(\text{dropl}(a, (x, y)))$ shows these hold only when the two tours agree on their first *and* second elements; the first is automatic for tours of one input, so

$$Q = R \cap (\text{next}^2(\text{next}^2)^\circ),$$

$$\text{next}([a] \# [b] \# x) = b.$$

As in §8.5, $P = R$ fails and $P = \Pi$ works, $\text{merge } \Pi = \text{cat}$. Binary thinning, with $\text{listcp} = \text{wrap} + \text{cpr}$:

$$\text{mintour} = (\text{start wrap, cpr } \langle \text{list dropl, list dropr} \rangle \text{ cat}(\text{thinlist } Q))(\text{minlist } R).$$

Ex 8.25 Exactly two tours are added per step, so the list of partial solutions grows linearly and the algorithm is

quadratic: after $[a_0, \dots, a_n]$ the next pairs are $(a_n, a_1), \dots, (a_2, a_1), (a_1, a_2), \dots, (a_1, a_n)$.

Ex 8.29 Longest upsequence: $\Lambda(\text{subseq ordered})(\max R)$ for $R = \text{length leq length}^\circ$ is another thinning algorithm.

9 Dynamic Programming

9.1 Theory

An optimal solution is composed of optimal solutions to subproblems — the *principle of optimality*. Decompose in every possible way, solve recursively, and assemble: that is Theorem 9.1. Discard the decompositions that can never contribute: that is Theorem 9.2. Discard all but one: that is chapter 10.

9.1 Throughout, $H = \llbracket T \rrbracket^\circ \llbracket h \rrbracket$ and $M = \Lambda(H)(\min R)$ — the problem being solved. h is a *map*: the chapter never needs the assembling algebra relational.

T 9.1 ✓ h monotonic on R — that is, $(FR)h \subset hR$ — and R transitive. Then

$$(\mu X : \Lambda(T^\circ) P((FX)h) (\min R)) \subset M.$$

Unfold T every way at once, solve each part, rebuild with h , keep a best. By Knaster–Tarski it is enough to make M a prefixed point:

$$(9.1) \checkmark \Lambda(T^\circ) P((FM)h) (\min R) \subset M.$$

which the universal property of $\min$ splits into a refinement and a bound:

$$(9.2) \checkmark \Lambda(T^\circ) P((FM)h) (\min R) \subset H.$$

$$(9.3) \checkmark H^\circ \Lambda(T^\circ) P((FM)h) (\min R) \subset R.$$

(9.4) ✓ Both use $(PX)(\min R) \subset (\exists X) \cap (\exists^\circ \setminus (XR))$ — which is (7.10), the two halves of fusion with the power functor.

$$\begin{aligned} & \Lambda(T^\circ) P((FM)h) (\min R) \\ \subset & \text{(9.4), member half} \\ & \Lambda(T^\circ) \exists (FM)h \\ = & \Lambda \text{ cancellation} \\ & T^\circ (FM)h \\ \subset & M \subset H, \text{ universal property of } \min \\ & T^\circ (FH)h \\ = & H \text{ and the hylomorphism theorem} \\ & H \end{aligned}$$

$$\begin{aligned} & H^\circ \Lambda(T^\circ) P((FM)h) (\min R) \\ \subset & \text{(9.4), lower-bound half} \\ & H^\circ \Lambda(T^\circ) (\exists^\circ \setminus ((FM)h R)) \\ = & H^\circ = h^\circ (FH^\circ) T, \text{ hylomorphism} \\ & h^\circ (FH^\circ) T \Lambda(T^\circ) (\exists^\circ \setminus ((FM)h R)) \\ \subset & X \Lambda(X^\circ) \subset \exists^\circ; \text{ division; functors} \\ & h^\circ F(H^\circ M)h R \\ \subset & M \subset H, \text{ universal property of } \min \\ & h^\circ (FR)h R \\ \subset & h \text{ monotonic, shunted} \\ & RR \\ \subset & R \text{ transitive} \\ & R \end{aligned}$$

T 9.2 ✓ **THINNING** h monotonic on R and Q a preorder with $Q^\circ (FH)h \subset (FH)h R^\circ$. Then

$$(\mu X : \Lambda(T^\circ)(\text{thin } Q) P((FX)h) (\min R)) \subset M.$$

The proviso says: throwing away the Q -worse decompositions throws away no optimum. The two conditions of dynamic programming are therefore

$$(FR)h \subset hR \quad \text{and} \quad Q^\circ (FH)h \subset (FH)h R^\circ.$$

Both theorems say the optimum is a *least* fixed point. Theorem 6.3 makes it the *only* one when $\text{mem}(F)T^\circ$ is inductive; and if $\Lambda(T^\circ)$ returns finite non-empty sets and R is a connected preorder, that solution is entire. Refining $\min R$ and $\Lambda(T^\circ)(\text{thin } Q)$ then gives a recursive function.

P 9.1 ✓ $F\alpha$ is usually a coproduct, so the recursion usually splits. If V_1, V_2 have disjoint ranges, $V_1 V_2^\circ = \emptyset$, then

$$\Lambda([V_1, V_2]^\circ)(\text{thin}(Q_1 + Q_2)) P([U_1, U_2]) (\min R) \\ (\text{ran } V_1 \rightarrow W_1, W_2),$$

where $W_i = \Lambda(V_i^\circ)(\text{thin } Q_i) (PU_i)(\min R)$. Every case study below uses it to peel off the base case.

Checking monotonicity

P 9.2 ✓ A cost function discharges it. If $R = \text{cost leq cost}^\circ$, $h \text{ cost} = (F \text{ cost})k$ and $(F \text{ leq})k \subset k \text{ leq}$, then $(FR)h \subset hR$.

$$\begin{aligned} & (FR)h \subset hR \\ \equiv & R; \text{shunting} \\ & (FR)h \text{ cost} \subset h \text{ cost leq} \\ \equiv & \text{assumption on cost} \\ & F(R \text{ cost})k \subset (F \text{ cost})k \text{ leq} \\ \Leftarrow & R \text{ cost} \subset \text{cost leq} \\ & F(\text{cost leq})k \subset (F \text{ cost})k \text{ leq} \\ \Leftarrow & \text{functors} \\ & (F \text{ leq})k \subset k \text{ leq} \\ \Leftarrow & k \text{ monotonic on leq} \\ & \text{true} \end{aligned}$$

P 9.3 ✓ **MONOTONICITY IN CONTEXT** the same with the answer carried alongside the cost: if $R = \text{cost leq cost}^\circ$, $h \text{ cost} = F(\text{cost}, H^\circ)k$, $F(\text{leq} \times 1)k \subset k \text{ leq}$ and H° is simple, then $F(R \cap (H^\circ H))h \subset hR$. Same chain as 9.2 with $\langle \text{cost}, H^\circ \rangle$ for cost; simplicity of H° is what lets the pair be cancelled.

P 9.4 ✓ **BOTH CONDITIONS AT ONCE** take F a bifunctor, written $F(1, X)$. If U, V are preorders with $F(U, R)h \subset hR$ and $V^\circ H \subset HR^\circ$, then Theorem 9.2 holds with $Q = F(U, V)$. Monotonicity is U reflexive; the rest:

$$\begin{aligned} & Q^\circ F(1, H)h \\ \equiv & Q = F(U, V); \text{converse; bifunctors} \\ & F(U^\circ, V^\circ H)h \\ \subset & \text{assumption on } V \\ & F(U^\circ, HR^\circ)h \\ \subset & \text{bifunctors} \\ & F(1, H)F(U^\circ, R^\circ)h \\ \subset & \text{assumption on } h, \text{converted and shunted} \\ & F(1, H)h R^\circ \end{aligned}$$

Ex 9.1 ✓ Theorem 9.1 is Theorem 9.2 at $Q = 1$.

Ex 9.2 ✓ **CONTEXT** both conditions weaken to $F(R \cap (H^\circ H))h \subset hR$ and $(Q \cap (TT^\circ)) (FH)h \subset (FH)h R^\circ$ — only decompositions of one and the same input need comparing.

Ex 9.4 ✓ $Q = F(MRM^\circ)$ always satisfies the thinning condition, and is useless: it presupposes the optimum M .

Ex 9.5 ✓ The three facts Proposition 9.1 rests on. Disjoint ranges is $\text{ran } V_2 = \sim(\text{ran } V_1) \text{ran } V_2$; then $\Lambda([V_1, V_2]^\circ) = (\text{ran } V_1 \rightarrow \Lambda(V_1^\circ \iota), \Lambda(V_2^\circ \iota'))$ and $(E\iota)(\text{thin}(Q_1 + Q_2)) = (\text{thin } Q_1)(E\iota)$, likewise for ι' .

9.2 The string edit problem

9.2 ✓ Turn one string into another by copying, deleting and inserting characters, in as few operations as possible. $\text{Op} ::= \text{cpy Char} \mid \text{del Char} \mid \text{ins Char}$, and $\text{edit} = (\text{base}, \text{step}) : \text{list Op} \rightarrow \text{list Char} \times \text{list Char}$ replays a sequence onto the two strings:

$$\text{base} = ([], []),$$

$$\text{step}(\text{cpy } a, (x, y)) = ([a] \# x, [a] \# y),$$

$$\text{step}(\text{del } a, (x, y)) = ([a] \# x, y),$$

$$\text{step}(\text{ins } a, (x, y)) = (x, [a] \# y).$$

Spec: a map $\text{mle} \subset \Lambda(\text{edit}^\circ)(\min R)$ with $R = \text{length leq length}^\circ$.

Monotonic: $\text{in} = [\text{nil}, \text{cons}]$ is monotonic under R , by Proposition 9.2 with $\text{length} = (\text{zero}, \pi' \text{ succ})$ and succ monotonic on leq .

Thinning: a copy, when available, beats a delete or an insert. With $F(A, B) = 1 + (A \times B)$ the condition is $Q^\circ F(1, \text{edit}^\circ) \text{ in} \subset F(1, \text{edit}^\circ) \text{ in } R^\circ$, and Proposition 9.4 reduces it to two:

$$F(\Pi, R) \text{ in} \subset \text{in } R \quad \text{and} \quad \text{edit } V \subset R \text{ edit}.$$

Ex 9.6 The first always holds, so $U = \Pi$: $(\Pi \times R)\text{cons} \subset \text{cons } R$.

For the second take $V = \text{suffix} \times \text{suffix}$. Since $\text{suffix} = \text{tail}^*$ and $AB \subset CB$ gives $A^*B \subset C^*B$, it is enough to have $\text{edit}(1 \times \text{tail}) \subset R \text{ edit}$ and $\text{edit}(\text{tail} \times 1) \subset R \text{ edit}$. In words: if $\text{edit } es = (x, \text{cons}(b, y))$, find the operation producing b and either demote $\text{cpy } b$ to $\text{del } b$ or delete $\text{ins } b$ — no longer, same result.

The recursion, with $Q = 1 + (U \times V)$:

$$X = \Lambda([\text{base}, \text{step}^\circ])(\text{thin } Q)$$

$$P([\text{nil}, (1 \times X)\text{cons}]) (\min R).$$

base and step have disjoint ranges, so Proposition 9.1 peels the base case:

$$X = (\text{empty} \rightarrow \text{nil}, \Lambda(\text{step}^\circ)(\text{thin}(U \times V)))$$

$$P((1 \times X)\text{cons})(\min R),$$

$\text{empty}(x, y)$ holding when both are $[]$.

$\Lambda(\text{step}^\circ)(\text{thin}(U \times V))$ is the list-valued unstep, and $\min R$ is $\text{minlist } R$:

$$\text{unstep}([a] \# x, []) = [(\text{del } a, (x, []))],$$

$$\text{unstep}([], [b] \# y) = [(\text{ins } b, ([], y))],$$

$$\text{unstep}([a] \# x, [a] \# y) = [(\text{cpy } a, (x, y))],$$

$$\text{unstep}([a] \# x, [b] \# y) =$$

$$[(\text{del } a, (x, [b] \# y)), (\text{ins } b, ([a] \# x, y))] \quad (a \neq b).$$

$$\text{mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle})\text{cons})(\text{minlist } R)).$$

It terminates — unstep shortens the combined length — and runs in exponential time, because the same subproblem is met again and again.

Tabulation

9.2 $\text{mle}(x, y)$ needs $\text{mle}(u, v)$ for every tail u of x and v of y : a grid, one column per tail of y , one row per tail of x . For $x = a_1 a_2 a_3$ and $y = b_1 b_2$:

$$\begin{array}{ccc} \text{mle}(a_1 a_2 a_3, b_1 b_2) & \text{mle}(a_1 a_2 a_3, b_2) & \text{mle}(a_1 a_2 a_3, []) \\ \text{mle}(a_2 a_3, b_1 b_2) & \text{mle}(a_2 a_3, b_2) & \text{mle}(a_2 a_3, []) \\ \text{mle}(a_3, b_1 b_2) & \text{mle}(a_3, b_2) & \text{mle}(a_3, []) \\ \text{mle}([], b_1 b_2) & \text{mle}([], b_2) & \text{mle}([], []) \end{array}$$

The leftmost column is column $x y$, the rightmost column $x []$, and the answer is the top of the leftmost. Each entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — so the columns can be built right to left, each from its neighbour:

$$\text{column } x = (\text{fstcol } x, \text{nextcol } x),$$

$$\text{fstcol} = (\text{list del})\text{tails}.$$

Written out, with m, n the lengths of $\text{mle}(u, [b] \# y)$ and $\text{mle}([a] \# u, y)$:

$$\text{mle}([a] \# u, [b] \# y) = \begin{cases} [\text{cpy } a] \# \text{mle}(u, y) & a = b \\ [\text{del } a] \# \text{mle}(u, [b] \# y) & m \leq n \\ [\text{ins } b] \# \text{mle}([a] \# u, y) & \text{else.} \end{cases}$$

$\text{nextcol } x(b, \text{us}) = (\text{base}(b, \text{last us}), \text{step } b) \ x \ \text{us over } x \ \text{us} = \text{zip}(x, \text{zip}(\text{init us}, \text{tail us}))$, with $\text{base}(b, u) = [[\text{ins } b] \# u]$ (a *different* base) and

$$\text{step } b((a, (u, v)), ws) =$$

$$\begin{cases} [[\text{cpy } a] \# v] \# ws & a = b \\ [\text{bmin } R([\text{del } a] \# w, [\text{ins } b] \# u)] \# ws & \text{else,} \end{cases}$$

$$w = \text{head } ws.$$

The program carries each sequence as a pair $(v, \text{length } v)$; column xy costs q calls of nextcol at $O(p)$ each, so $O(pq)$ — quadratic.

9.3 Optimal bracketing

9.3 ✓ Bracket $a_1 \oplus \dots \oplus a_n$ as cheaply as possible. tree $A := \text{tip } A \mid \text{bin}(\text{tree } A, \text{tree } A)$ and $\text{flatten} = (\text{wrap}, \text{cat}) : \text{tree } A \rightarrow \text{list}^+ A$ reads off the tips left to right. Spec: a map $\text{mct} \subset \Lambda((\text{wrap}, \text{cat})^\circ)(\text{min } R)$ with $R = \text{cost leq cost}^\circ$.

The cost of a tip is zero; the cost of a node is a function cb of the *sizes* of its two subtrees, plus their costs. size is not the number of elements:

$$\text{cost}(\text{bin}(x, y)) = \text{cb}(\text{size } x, \text{size } y) + \text{cost } x + \text{cost } y,$$

$$\text{size}(\text{bin}(x, y)) = \text{sb}(\text{size } x, \text{size } y),$$

$$\langle \text{cost}, \text{size} \rangle = (\text{opt}, \text{opb}),$$

$$\text{opt} = \langle \text{zero}, \text{st} \rangle,$$

$$\text{opb}((c_x, s_x), (c_y, s_y)) = (\text{cb}(s_x, s_y) + c_x + c_y, \text{sb}(s_x, s_y)).$$

Rel $x_1 \# \dots \# x_n$ on cons-lists: $\text{cb}(m, n) = m$, $\text{sb}(m, n) = m + n$, $\text{st} = \text{length}$. Bracketing to the right is always optimal — which is why concat is a cons-list catamorphism.

No decomposition is obviously better than another, so there is no thinning step and Theorem 9.1 is the target. The one assumption: *size depends only on the atoms, not on the bracketing* — true when sb is associative, and then $\text{size} = \text{flatten } \text{sz}$ for a map sz .

Monotonicity comes from Proposition 9.3 with $g = [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi]$:

(9.5) $F(1 + \langle \text{cost}, \text{flatten} \rangle^2)g = [\text{tip}, \text{bin}] \text{ cost} - g$ is Proposition 9.3's k . Unfold g , use $\text{flatten } \text{sz} = \text{size}$, then $\langle \text{cost}, \text{size} \rangle = (\text{opt}, \text{opb})$ and $\text{tip cost} = \text{zero}$.

(9.6) $(1 + (\text{leq} \times 1)^2)g \subset g \text{ leq}$ — from the shape of opb , monotonicity of $+$, and reflexivity of leq .

Theorem 9.1 and then Proposition 9.1, wrap and cat having disjoint ranges:

$$X = \Lambda([\text{wrap}, \text{cat}]^\circ) P([\text{tip}, (X \times X) \text{bin}]) (\text{min } R),$$

$$X = (\text{single} \rightarrow \text{wrap}^\circ \text{ tip},$$

$$\Lambda(\text{cat}^\circ) P((X \times X) \text{bin})(\text{min } R)).$$

$\Lambda(\text{cat}^\circ)$ is $\text{splits} = \langle \text{inits}^+, \text{tails}^+ \rangle \text{ zip}$, the proper prefixes against the proper suffixes, giving

$$\text{mct} = (\text{single} \rightarrow \text{head tip},$$

$$\text{splits list}((\text{mct} \times \text{mct}) \text{bin})(\text{minlist } R)).$$

Both halves of a split are shorter, so it terminates; and it is exponential.

$\text{row} = \text{tails list mct}$, $\text{col} = \text{inits list mct}$. For $x = a_1 a_2 a_3 a_4$:

$$\begin{array}{cccc} \text{mct}(a_1) & & & \\ \text{mct}(a_1 a_2) & \text{mct}(a_2) & & \\ \text{mct}(a_1 a_2 a_3) & \text{mct}(a_2 a_3) & \text{mct}(a_3) & \\ \text{mct}(a_1 a_2 a_3 a_4) & \text{mct}(a_2 a_3 a_4) & \text{mct}(a_3 a_4) & \text{mct}(a_4) \end{array}$$

The answer is the bottom of the leftmost column. Three mutual recursions, each got by pushing list mct through the definition of inits or tails :

(9.7) $\text{mct} = (\text{single} \rightarrow \text{head tip}, \langle \text{init col}, \text{tail row} \rangle \text{ mix})$ with $\text{mix} = \text{zip list bin}(\text{minlist } R)$.

(9.8) $\text{col} = (\text{single} \rightarrow \text{head tip wrap}, \langle \text{init col}, \text{tail row} \rangle \text{ next})$ with $\text{next} = \langle \pi, \text{mix} \rangle \text{ snoc}$, so a column is a loop.

(9.9) $\text{cons col} = (1 \times \text{array}) \text{ process}$ with $\text{process} = ((\text{tip wrap}) \times 1) (\text{loop next})$ — the column of $\text{cons}(a, x)$ by looping next over the array of x .

(9.10) $\text{row} = (\text{single} \rightarrow \text{head tip wrap}, \langle \text{mct}, \text{tail row} \rangle \text{ cons})$.

Hence $\text{array} = (\text{fstcol}, \text{addcol})$ with $\text{fstcol} = \text{tip wrap wrap}$ and

$$\text{addcol} = \langle \pi \text{ tip wrap}, \text{step} \rangle \text{ cons},$$

$$\text{step} = \langle \text{process tail}, \pi' \rangle \text{ zip list cons}.$$

$\text{mct } x = (\text{array last head})x$, each node labelled with its cost and size. addcol runs $n - 1$ times and step costs $O(m^2)$ on an $m \times m$ array, so $O(n^3)$.

Ex 9.12 h associative $\implies (\text{wrap}, \text{cat}) (\text{wrap } h) = (\text{wrap } h)$, the left one over trees and the right over non-empty lists.

Ex 9.13 $k = (g \times \text{inits list } h)(\text{loop } f)$ exactly when $(1 \times \text{nil})k = \pi g$ and $(1 \times \text{snoc})k = \langle (1 \times \pi)k, \pi' \text{ snoc } h \rangle f$. This is what proves (9.9).

9.4 Data compression

9.4 ✓ Compress by textual substitution: a code element is a character or a pointer back into the part already decoded, $\text{Code} := \text{sym Char} \mid \text{ptr}(\text{String}, \text{String}^+)$. Snoc -lists here. $\text{decode} = (\text{nil}, \text{extend})$ is a *partial* map:

$$\text{extend}(x, \text{sym } a) = x \# [a],$$

$$\text{extend}(x, \text{ptr}(y, z)) = x \# z, \quad (y \# z) \text{init}^+(x \# z).$$

$y \# z$ need not be a prefix of x , so a pointer may overlap what it produces: $\text{extend}(\text{aba}, \text{ptr}(\text{a}, \text{bab})) = \text{ababab}$. A leading pointer is undefined, since nothing precedes it; but decode is surjective, so everything can be coded. Compactly, a pointer is a pair of lengths: $x \otimes (m, 0) = x$ and $x \otimes (m, n + 1) = (x \# [x_m]) \otimes (m + 1, n)$.

Rel $\text{decode}[a', (0, 9)] = \text{aaaaaaaaa}$ and $\text{decode}[a', a', b', (1, 3), c', (1, 2)] = \text{aababacab}$.

$\text{size} = (\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus})$ counts bytes, c and p constants set by the machine. Spec: a map $\text{encode} \subset \Lambda(\text{decode}^\circ)(\text{min } R)$ with $R = \text{size leq size}^\circ$.

Monotonicity is routine; the thinning is the work. Symbol against pointer cannot be decided for general c, p , but pointer against pointer can: a pointer consuming more input is better. If $w = \text{extend}(x, \text{ptr}(y, z)) = \text{extend}(x', \text{ptr}(y', z'))$ then z is longer than z' exactly when x is a prefix of x' . So take

$$Q = F(\Pi + \Pi, \text{prefix}),$$

$$F(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}),$$

and Proposition 9.4 asks for $F(\Pi + \Pi, R)$ in $\subset$ in R (routine: symbol and pointer sizes are constants, so $(\Pi + \Pi)[c, p] = [c, p]$ — the book prints d for p here)

Tabulation

9.3 $\text{mct } x$ needs $\text{mct } y$ for every non-empty segment y of x : a triangle, held as a list of rows. $\text{array} = \text{inits list row}$,

and $\text{decode prefix} \subset R \text{ decode}$, which follows from $\text{decode init} \subset R \text{ decode}$: drop a trailing sym a , or shorten a trailing $\text{ptr}(y, z \# [a])$ to $\text{ptr}(y, z)$, or drop it when $z = []$.

The recursion, with $U = \text{prefix}$ and $V = \Pi + \Pi$, then Proposition 9.1 on the disjoint ranges of nil and extend :

$$X = \Lambda([\text{nil}, \text{extend}]^\circ)(\text{thin}(1 + (U \times V)))$$

$$P([\text{nil}, (X \times 1)\text{snoc}](\min R),$$

$$X = (\text{null} \rightarrow \text{nil}, \Lambda(\text{extend}^\circ)(\text{thin}(U \times V)))$$

$$P((X \times 1)\text{snoc})(\min R)).$$

$\Lambda(\text{extend}^\circ)(\text{thin}(U \times V))$ is reduce , built on the *longest repeated tail*

$$\text{lrt } w = \min(U \times V)$$

$$\{(x, (y, z)) \mid x \# z = w \wedge (y \# z)\text{init}^+ w\},$$

$$\text{reduce}(w \# [a]) = \begin{cases} [(w, \text{sym } a), (x, \text{ptr}(y, z))] & z \neq [] \\ [(w, \text{sym } a)] & \text{else,} \end{cases}$$

$$(x, (y, z)) = \text{lrt}(w \# [a]).$$

$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1)\text{snoc})(\text{minlist } R))$. The same subproblem recurs, so a tabulation over the initial segments is called for; the book leaves the details, calling them messy.

10 Greedy Algorithms

10.1 Theory

Weed out all but a *single* decomposition at each step and dynamic programming becomes greedy. The theory is chapter 9's; the chapter is its applications.

^{10.1 ✓} **THE GREEDY THEOREM** $H = \langle T \rangle^\circ \langle h \rangle$ and $M = \Lambda(H)(\min R)$, h monotonic on R , and Q with $Q^\circ(FH)h \subset (FH)h R^\circ$ — exactly Theorem 9.2's hypotheses. Then

$$(\mu X : \Lambda(T^\circ)(\min Q)(FX)h) \subset M.$$

$\min Q$ where Theorem 9.2 had $\text{thin } Q$: commit to one decomposition instead of keeping a thinned set. Same hypotheses, far stronger conclusion — and the price is a further, very strong condition for the refinement to a program: Q must be a *connected* preorder on the sets $\Lambda(T^\circ)$ returns. Distinct from Theorem 7.2, which greedily folds a catamorphism.

^{P 10.1 ✓} $F\alpha$ is usually a coproduct. If $V_1 V_2^\circ = \emptyset$ then

$$\Lambda([V_1, V_2]^\circ)(\min(Q_1 + Q_2))[U_1, U_2]$$

$$= (\text{ran } V_1 \rightarrow W_1, W_2),$$

$$W_i = \Lambda(V_i^\circ)(\min Q_i)U_i.$$

This is the step that turns each case study's fixed point into a conditional program.

Proposition 9.4 still supplies $Q = F(U, V)$ from preorders with $F(U, R)h \subset hR$ and $V^\circ H \subset HR^\circ$ — but now $\Lambda(T^\circ)(\min Q)$ must also be *entire*, which that choice does not guarantee.

10.2 The *detab*–*entab* problem

^{10.2 ✓} Two exercises of Kernighan and Ritchie. *detab* replaces each tab by blanks up to the next stop, every n columns; *entab* replaces runs of blanks by the fewest tabs and blanks giving the same spacing. They belong together because *entab* is specified as an optimal converse of *detab*.

Snoc-lists. $\text{detab} = \langle \text{nil}, \text{expand} \rangle$ with

$$\text{expand}(x, a) = (a = \text{TB} \rightarrow \text{fill } x, x \# [a]),$$

$$\text{fill } x = x \# \text{blanks}(n - (\text{col } x) \bmod n),$$

$$\text{col} = \langle \text{zero}, \text{count} \rangle,$$

$$\text{count}(c, a) = (a = \text{NL} \rightarrow 0, c + 1).$$

Tupling keeps the column count: $\langle \text{detab}, \text{detab col} \rangle = \langle \text{base}, \text{step} \rangle$ with $\text{base} = \langle [], 0 \rangle$ and, writing $m = n - c \bmod n$,

$$\text{step}((x, c), a) = \begin{cases} (x \# [\text{NL}], 0) & a = \text{NL} \\ (x \# \text{blanks } m, c + m) & a = \text{TB} \\ (x \# [a], c + 1) & \text{else.} \end{cases}$$

^{Ex 10.1} **LOOP** a snoc-list catamorphism is a loop over the cons-list: convert $\langle \text{base}, \text{step} \rangle = \langle \text{base}, 1 \rangle(\text{loop step})$. And when the two have the special shape $\text{base} = \langle \text{nil}, c_0 \rangle$ and $\text{step}((x, c), a) = (x \# f(c, a), g(c, a))$, the accumulator can go:

$$\langle \text{base}, 1 \rangle(\text{loop step}) \pi = \langle c_0, 1 \rangle(\text{loop}'(f, g)),$$

$$\text{loop}'(f, g)(c, []) = [],$$

$$\text{loop}'(f, g)(c, [a] \# x)$$

$$= f(c, a) \# \text{loop}'(f, g)(g(c, a), x).$$

Why it pays: the left side builds $((x_1 \# x_2) \# \dots) \# x_n$ and the right side $x_1 \# (x_2 \# (\dots \# x_n))$. Where $\#$ is defined by cons, the second is asymptotically cheaper.

entab is a shortest output that decodes back:
 $\text{entab} \subset \Lambda(\text{detab}^\circ)(\min R)$ with $R = \text{length leq length}^\circ$.

nil and expand have disjoint ranges, so look for $Q = F(U, V)$ with $F(U, V) = 1 + (V \times U)$. Proposition 9.4 asks for

$$F(U, R) \text{ in } \subset \text{ in } R \quad \text{and} \quad \text{detab } V \subset R \text{ detab},$$

and, on top, $\Lambda([\text{nil}, \text{expand}]^\circ)(\min Q)$ must be entire.

Ex 10.3 The first holds for *every* preorder U on characters, since $F(\Pi, R) \text{ in } \subset \text{ in } R$. Taking $U = 1 \cup (\Pi \text{ TB})$ — everything above the tab — prefers tabs to blanks.
 $V = \text{prefix fails}$. Rel at $n = 8$, $\text{detab}[a, b, c, d, e, \text{TB}] = [a, b, c, d, e, \text{BL}, \text{BL}, \text{BL}]$, and the prefix $[a, b, c, d, e, \text{BL}, \text{BL}]$ is longer than $[a, b, c, d, e, \text{TB}]$.
 Allow only prefixes that do not cross a tab stop: $V = \text{prefix} \cap (\text{fill fill}^\circ)$.

$\text{detab } V$
 $=$ detab is a catamorphism
 $[\text{nil}, \text{snoc}]^\circ F(1, \text{detab})[\text{nil}, \text{expand}]V$
 $=$ coproducts, and $\text{nil } V = \text{nil}$
 $[\text{nil}, \text{snoc}]^\circ F(1, \text{detab})[\text{nil}, \text{expand}]V$
 $\subset$ claim: $\text{expand } V \subset \text{expand} \cup (\pi V)$
 $[\text{nil}, \text{snoc}]^\circ F(1, \text{detab})[\text{nil}, \text{expand} \cup (\pi V)]$
 $=$ distribute $\cup$; catamorphisms; F
 $\text{detab} \cup (\text{snoc}^\circ(\text{detab} \times 1)\pi V)$
 $=$ naturality of π , and $\text{init} = \text{snoc } \pi$
 $\text{detab} \cup (\text{init detab } V)$

So $X = \text{detab } V$ solves $X \subset \text{detab} \cup (\text{init } X)$. init is inductive, so the greatest solution of the inequation is the unique solution of the equation, namely $X = \text{prefix detab}$; and $\text{prefix} \subset R$.

Ex 10.4 The claim rests on $\text{nil } V = \text{nil}$, $\text{fill } V = \text{fill}$ and $\text{snoc } V \subset \text{snoc} \cup (\pi V)$ — the last is the one $V = \text{prefix}$ breaks.

The greedy theorem now applies, and Proposition 10.1 peels the base case:

$$X = \Lambda([\text{nil}, \text{expand}]^\circ)(\min Q)(1 + (X \times 1))[\text{nil}, \text{snoc}],$$

$$Q = 1 + (V \times U),$$

$$X = (\text{null} \rightarrow \text{nil}, \Lambda(\text{expand}^\circ)(\min(V \times U))(X \times 1)\text{snoc}).$$

$\Lambda(\text{expand}^\circ)(x \# [a]) = \{(y, \text{TB}) \mid \text{fill } y = x \# [a]\} \cup \{(x, a)\}$, and the first set is non-empty exactly when $a = \text{BL}$ and $\text{col}(x \# [a]) \bmod n = 0$. Its V -minimum is $(\text{unfill } x, \text{TB})$, where $\text{unfill } x$ is the shortest prefix with $\text{fill}(\text{unfill } x) = \text{fill } x$:

$$\text{unfill } [] = [],$$

$$\text{unfill}(x \# [a]) = \begin{cases} \text{unfill } x & a = \text{BL} \wedge \neg \text{stop} \\ x \# [a] & \text{else,} \end{cases}$$

writing stop for the test that the new column is a tab stop, $\text{col}(x \# [a]) \bmod n = 0$, equivalently $(c + 1) \bmod n = 0$ for c the column count.

(10.1) $\text{entab } x = \text{entab}(\text{unfill } x) \# \text{blanks}(\text{tbc } x)$, with tbc the count of trailing blanks. This is what turns entab into a snoc -list catamorphism:

$$\text{tbc } [] = 0,$$

$$\text{tbc}(x \# [a]) = \begin{cases} 0 & a = \text{BL} \wedge \text{stop} \\ \text{tbc } x + 1 & \text{else,} \end{cases}$$

$\langle \text{tbc}, \text{col} \rangle$ is a catamorphism, and so is the triple $\text{triple} = \langle \text{unfill entab}, \langle \text{tbc}, \text{col} \rangle \rangle = \langle \text{base}, \text{op} \rangle$ from $([], (0, 0))$ with

$$\text{op}((x, (t, c)), a) = \begin{cases} (x, (t + 1, c + 1)) & a = \text{BL} \wedge \neg \text{stop} \\ (x \# [\text{TB}], (0, c + 1)) & a = \text{BL} \wedge \text{stop} \\ (x \# \text{blanks } t \# [\text{NL}], (0, 0)) & a = \text{NL} \\ (x \# \text{blanks } t \# [a], (0, c + 1)) & \text{else,} \end{cases}$$

$$\text{entab} = \text{triple assocl } \pi (1 \times \text{blanks}) \text{ cat},$$

and then loop' applies to triple exactly as it did to detab .

10.3 The minimum tardiness problem

10.3 ✓ Given a *bag* of jobs, find a permutation of it — a *schedule* — minimising the largest penalty for lateness. Each job j has a completion time $\text{ct } j$, a due time $\text{dt } j$ and a weight $\text{wt } j$, all positive, and $\text{penalty}(x, j) = (\text{sum}(\text{list } \text{ct } x) + \text{ct } j - \text{dt } j) \times \text{wt } j$. Bonuses do not count: the cost is the largest penalty incurred, never negative.

$\text{schedule} \subset \Lambda(\text{bagify}^\circ)(\min R)$ with $R = \text{cost leq cost}^\circ$, $\text{bagify} = \langle \text{nil}, \text{snag} \rangle$ (snag drops a job into a bag), and

$$\text{cost} = \Lambda(\text{prefix}) P(\text{in}^\circ[\text{zero}, \text{penalty}]) (\text{max leq}),$$

$$\text{cost}(x \# [j]) = \text{bmax}(\text{cost } x, \text{penalty}(x, j)).$$

	1	2	3		2	3	1
ct	5	10	15	time	10	25	30
dt	10	20	20	dt	20	20	10
wt	1	3	3	penalty	0	15	20

Rel three jobs, and the walk-through of $[2, 3, 1]$ — one of the two best schedules, cost 20, the other being $[3, 2, 1]$.

The algorithm will pick the job best placed *last*: a **BACKWARD GREEDY** algorithm, which is why snoc -lists. Taking the input as a list instead of a bag, $\text{schedule} \subset \Lambda(\text{perm})(\min R)$ with $\text{perm} = \langle \text{nil}, \text{add} \rangle$, where $\text{add}(x, j) = y \# [j] \# z$ for any split $x = y \# z$.

Chapter 7's method does not reach this problem. A snoc -list catamorphism solves every *prefix* of the input too, and Rel for $[1, 2, 3]$ the best schedule of the prefix $[1, 2]$ is $[1, 2]$, at cost zero, which extends to neither $[2, 3, 1]$ nor $[3, 2, 1]$. Context has to be brought into both conditions:

(10.2) $F(R \cap (\text{bagify } \text{bagify}^\circ)) \text{ in } \subset \text{ in } R$, with $\text{in} = [\text{nil}, \text{snoc}]$ and $FX = 1 + (X \times \text{Job})$.

(10.3) $(Q^\circ \cap (\beta\beta^\circ))F(\text{bagify}^\circ) \text{ in } \subset F(\text{bagify}^\circ) \text{ in } R^\circ$, with $\beta = [\text{nil}, \text{snag}]$.

For (10.2), $\text{penalty}(x, j)$ depends only on the jobs of x , not their order, so $\text{cost}(x \# [j]) = \text{bmax}(\text{cost } x, \text{penalty}(\text{perm } x, j))$ and Proposition 9.3 applies with

$$\text{in cost} = F\langle \text{cost}, \text{bagify} \rangle k,$$

$$k = [\text{zero}, \text{assocl } (1 \times ((\text{bagify}^\circ \times 1)\text{penalty})) \text{ bmax}].$$

(10.4) $\text{in cost} = \langle g, h \rangle \text{ bmax}$, the original cost resplit,

(10.5) $g = [\text{zero}, \text{penalty}]$,

(10.6) $h = [\text{zero}, \pi \text{ cost}]$.

(10.7) $\text{add} \subset \pi R^\circ$ — adding a job can only raise the cost.

(10.8) $\beta \text{ bagify}^\circ = F(\text{bagify}^\circ)[\text{nil}, \text{add}]$ — bagify° is a catamorphism on bags.

$$\beta \text{ bagify}^\circ$$

$=$ (10.8) $F(\text{bagify}^\circ)[\text{nil}, \text{add}]$
 $\subset$ (10.7) $F(\text{bagify}^\circ)[\text{nil}, \pi R^\circ]$
 $\subset$ R ; and $\text{nil} \subset \text{zero geq cost}^\circ$
 $F(\text{bagify}^\circ)[\text{zero}, \pi \text{cost}] \text{ geq cost}^\circ$
 $=$ (10.6) $F(\text{bagify}^\circ)h \text{ geq cost}^\circ$
 $(Q^\circ \cap (\beta\beta^\circ))F(\text{bagify}^\circ) \text{ in}$
 $\subset$ composition is monotonic
 $(Q^\circ F(\text{bagify}^\circ) \text{ in}) \cap (\beta\beta^\circ F(\text{bagify}^\circ) \text{ in})$
 $=$ $\text{bagify} = \langle \beta \rangle$
 $(Q^\circ F(\text{bagify}^\circ) \text{ in}) \cap (\beta \text{ bagify}^\circ)$
 $\subset$ the chain above
 $(Q^\circ F(\text{bagify}^\circ) \text{ in}) \cap (F(\text{bagify}^\circ)h \text{ geq cost}^\circ)$
 $\subset$ modular law
 $F(\text{bagify}^\circ)((F(\text{bagify}^\circ)Q^\circ F(\text{bagify}^\circ)) \cap$
 $(h \text{ geq cost}^\circ \text{ in}^\circ)) \text{ in}$
 $=$ choose Q with $F(\text{bagify}^\circ)Q^\circ F(\text{bagify}^\circ) = g \text{ geq } g^\circ$
 $F(\text{bagify}^\circ)((g \text{ geq } g^\circ) \cap (h \text{ geq cost}^\circ \text{ in}^\circ)) \text{ in}$
 $=$ products
 $F(\text{bagify}^\circ)\langle g \text{ geq}, h \text{ geq} \rangle \langle g, \text{in cost}^\circ \rangle^\circ \text{ in}$

$Q = f \text{ leq } f^\circ$ with $f = [\text{zero}, (\text{bagify}^\circ \times 1)\text{penalty}]$ meets that specification: a Q -minimum is a job of least penalty. What is left, after shunting cost across, is:

$\langle g \text{ geq}, h \text{ geq} \rangle \langle g, \text{in cost}^\circ \rangle^\circ \text{ in cost}$
 $\subset$ (10.4), and $\langle g, \text{incost} \rangle \text{ simple}$
 $\langle g \text{ geq}, h \text{ geq} \rangle \text{ bmax}$
 $\subset$ bmax monotonic
 $\langle g, h \rangle \text{ bmax geq}$
 $=$ (10.4)
 in cost geq

The greedy theorem applies, and Proposition 10.1 gives the program:

$X = \Lambda([\text{nil}, \text{snag}^\circ](\min Q)(1 + (X \times 1))[\text{nil}, \text{snoc}],$
 $\text{schedule} = (\text{null} \rightarrow \text{nil}, \text{pick}(\text{schedule} \times 1) \text{ snoc}),$

where pick refines $\Lambda(\text{snag}^\circ)(\min Q')$ with $Q' = f' \text{ leq } f'^\circ$, $f' = (\text{bagify}^\circ \times 1)\text{penalty}$. (The book prints both f and f' as f .) Bags are lists paired with $\text{sum}(\text{list ct } x)$; pick takes the first job of least penalty, and the program is quadratic.

Ex 10.7 add is itself a least fixed point: $\text{add} = (\mu X : \text{snoc} \cup ((\text{snoc} \times 1)\text{exch}(X \times 1)\text{snoc}))$, with $\text{exch} : (A \times B) \times C \rightarrow (A \times C) \times B$. Fixed-point induction plus $(\text{add} \times 1)\text{penalty} \subset (\pi \times 1) \text{ penalty geq}$ then proves (10.7).

Ex 10.8 $\text{perm} = \text{bagify bagify}^\circ = \langle \text{nil}, \text{add} \rangle$ proves (10.8).

10.4 The TeX problem – part two

10.4 ✓ Convert between decimal fractions and integer multiples of 2^{-16} , in integer arithmetic only. $\text{extern} \subset \Lambda(\text{intern}^\circ)(\min R)$ with $R = \text{length leq length}^\circ$ and

$\text{intern} = \text{val round},$
 $\text{round } r = \lfloor 2^{16}r + 1/2 \rfloor,$
 $\text{val} = \langle \text{zero}, \text{shift} \rangle,$
 $\text{shift}(d, r) = (d + r)/10.$

round° is not a map, so it cannot leave the Λ . But $n = \lfloor 2^{16}r + 1/2 \rfloor \iff 2n - 1 \leq 2^{17}r < 2n + 1$, which factors it through a map:

$\text{round}^\circ = \text{interval inrange},$
 $\text{interval } n = ((2n - 1)/2^{17}, (2n + 1)/2^{17}),$

$(a, b) \text{ inrange } r \iff a \leq r < b,$

$\text{extern} \subset \text{interval } \Lambda(\text{inrange val}^\circ)(\min R).$

Fusion makes val inrange° a catamorphism $\langle \text{arb}, \text{step} \rangle$: the conditions are $\text{zero inrange}^\circ = \text{arb}$ and $\text{shift inrange}^\circ = (1 \times \text{inrange}^\circ)\text{step}$, and the second unfolds pointwise to $\text{step}(d, (a', b')) = ((d + a')/10, (d + b')/10)$.

(10.9) Every interval reached satisfies $0 < b < 1$ and $a < b$ — true of interval n for $0 \leq n < 2^{16}$, preserved by step, and impossible on arb . So $[\text{arb}, \text{step}] : 1 + (\text{Digit} \times \text{Interval}) \rightarrow \text{Interval}$, and the derivation lives inside that type.

$\text{in} = [\text{nil}, \text{cons}]$ is monotonic under R , leaving the greedy condition: a Q over $F \text{ Interval}$, $FA = 1 + (\text{Digit} \times A)$, with $\text{in}^\circ(Fh)Q \subset R \text{ in}^\circ(Fh)$ for $h = \langle \text{arb}, \text{step} \rangle$.

Q need only choose between two things, because (10.9) makes step° a map:

$\Lambda(\text{arb}^\circ)(a, b) = (a \leq 0 \rightarrow \{*, \{\}\},$

$\Lambda(\text{step}^\circ)(a, b)$

$= \{(d, (10a - d, 10b - d)) \mid 0 < 10b - d < 1\},$

and $0 < 10b - d_1 < 1$ with $0 < 10b - d_2 < 1$ forces $d_1 = d_2$, so $\text{step}^\circ(a, b) = (d, (10a - d, 10b - d))$ at $d = \lfloor 10b \rfloor$. Hence $\Lambda([\text{arb}, \text{step}]^\circ)(a, b)$ is $\{\iota *, \iota'(d, \dots)\}$ when $a \leq 0$ and $\{\iota'(d, \dots)\}$ otherwise, and

$Q = (\iota^\circ ! \iota) \cup 1,$

$! : \text{Digit} \times \text{Interval} \rightarrow 1,$

which prefers the terminal inhabitant — stop — whenever stopping is possible.

The greedy proof is nine shuntings around one inequation. Unfold Q ; drop the 1 disjunct (R reflexive); use the universal property of $!$ with $\iota' \text{ in} = \text{cons}$ and $\iota \text{ in} = \text{nil}$ to reduce everything to $! \text{nil} \subset \text{cons } R$, which is $! \text{nil length} \subset \text{cons length leq}$: the empty decimal is no longer than any other.

So compute the least solution of $X = \Lambda([\text{arb}, \text{step}]^\circ)(\min Q)(FX) \text{ in}$, which simplifies to $\text{extern} = \text{interval } f$ with

$$f(a, b) = \begin{cases} \emptyset & a \leq 0 \\ [d] \# f(10a - d, 10b - d) & \text{else,} \end{cases}$$

$$d = \lfloor 10b \rfloor.$$

Integers last: every interval reached is $(p/w, q/w)$ for $w = 2^{17}$, since interval $n = ((2n - 1)/w, (2n + 1)/w)$ and $10a - d = (10p - wd)/w$. Representing it by (p, q) : $\text{extern } n = f(2n - 1, 2n + 1)$ with

$$f(p, q) = \begin{cases} \emptyset & p \leq 0 \\ [d] \# f(10p - wd, 10q - wd) & \text{else,} \end{cases}$$

$$d = (10q) \text{ div } w.$$

Ex 10.14 The only property of $w = 2^{17}$ used is that $10q/w$ is never an integer for $0 < q < w$.

Ex 10.15 Knuth asked for the shortest and closest decimal to $n/2^{16}$. This algorithm delivers only the shortest; which one it picks, and how to get the closest, the book leaves open.